

ARM PrimeCell™

Static Memory Controller (PL092)

Technical Reference Manual



ARM PrimeCell™ Static Memory Controller (PL092)

Technical Reference Manual

Copyright © ARM Limited 2001. All rights reserved.

Release information

Change history

Date	Issue	Change
April 2001	A	First draft.
June 2001	B	Signal changes to F1-3, Page 2-58, A-7, A-8.

Proprietary notice

Words and logos marked with ® or ™ are registered trademarks or trademarks owned by ARM Limited, except as otherwise stated below in this proprietary notice. Other brands and names mentioned herein may be the trademarks of their respective owners.

All other products or services mentioned herein may be trademarks of their respective owners.

Neither the whole nor any part of the information contained in, or the product described in, this document may be adapted or reproduced in any material form except with the prior written permission of the copyright holder.

The product described in this document is subject to continuous developments and improvements. All particulars of the product and its use contained in this document are given by ARM Limited in good faith. However, all warranties implied or expressed, including but not limited to implied warranties or merchantability, or fitness for purpose, are excluded.

This document is intended only to assist the reader in the use of the product. ARM Limited shall not be liable for any loss or damage arising from the use of any information in this document, or any error or omission in such information, or any incorrect use of the product.

Document confidentiality status

This document is Open Access. This means there is no restriction on the distribution of the information.

Product status

The information in this document is Final (information on a developed product).

ARM web address

<http://www.arm.com>

Contents

ARM PrimeCell™ Static Memory Controller (PL092) Technical Reference Manual

	Preface	
	About this document	vi
	Further reading.....	ix
	Feedback	x
Chapter 1	Introduction	
	1.1 About the ARM PrimeCell Static Memory Controller (PL092).....	1-2
Chapter 2	Functional Overview	
	2.1 ARM PrimeCell SMC overview	2-2
	2.2 PrimeCell SMC operation.....	2-6
	2.3 Memory shadowing	2-46
	2.4 Test interface controller operation.....	2-48
	2.5 Data bus interface operation.....	2-58
	2.6 Using the SMC with an external bus multiplexer or SDRAM controller.....	2-60
Chapter 3	Programmer's Model	
	3.1 About the programmer's model.....	3-2
	3.2 PrimeCell SMC registers.....	3-3
Chapter 4	Programmer's Model for Test	
	4.1 Scan testing	4-2

Appendix A

ARM PrimeCell Static Memory Controller (PL092) Signal Descriptions

A.1

AMBA AHB interface signals

A-2

A.2

AMBA AHB slave interface signals.....

A-3

A.3

AMBA AHB master interface signals

A-5

A.4

Internal signals

A-7

A.5

Input/output pad signals

A-9

Index

Preface

This preface introduces the ARM PrimeCell Static Memory Controller (PL092) and its reference documentation. It contains the following sections:

- *About this document* on page vi
- *Further reading* on page ix
- *Feedback* on page x.

About this document

This document is the technical reference manual for the ARM PrimeCell *Static Memory Controller* (SMC).

Intended audience

This document has been written for implementation engineers and architects, and provides a description of an optimal PrimeCell SMC architecture. The PrimeCell SMC provides an interface between the *Advanced High-performance Bus* (AHB) system bus and external (off-chip) memory devices.

Using this manual

This document is organized into the following chapters:

Chapter 1 *Introduction*

Read this chapter for an introduction to the PrimeCell SMC and its features.

Chapter 2 *Functional Overview*

Read this chapter for an overview of the major functional blocks and the operation of the PrimeCell SMC.

Chapter 3 *Programmer's Model*

Read this chapter for a description of the registers and for details of system initialization.

Chapter 4 *Programmer's Model for Test*

Read this chapter for a description of the additional logic for functional verification and production testing.

Appendix A *ARM PrimeCell SMC (PL092) Signal Descriptions*

Read this appendix for a description of the PrimeCell SMC signals.

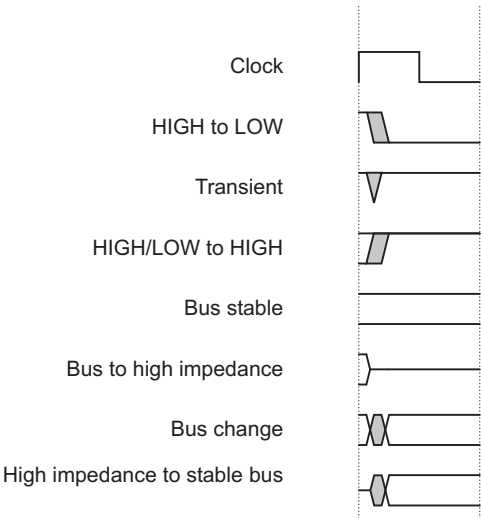
Typographical conventions

The following typographical conventions are used in this document:

bold	Highlights ARM processor signal names, and interface elements such as menu names. Also used for terms in descriptive lists, where appropriate.
<i>italic</i>	Highlights special terminology, cross-references, and citations.
typewriter	Denotes text that can be entered at the keyboard, such as commands, file names and program names, and source code.
<u>typewriter</u>	Denotes a permitted abbreviation for a command or option. The underlined text can be entered instead of the full command or option name.
<i>typewriter italic</i>	Denotes arguments to commands or functions where the argument is to be replaced by a specific value.
typewriter bold	Denotes language keywords when used outside example code.

Timing diagram conventions

This manual contains one or more timing diagrams. The following key explains the components used in these diagrams. Any variations are clearly labeled when they occur. Therefore, no additional meaning should be attached unless specifically stated.



Key to timing diagram conventions

Shaded bus and signal areas are undefined, so the bus or signal can assume any value within the shaded area at that time. The actual level is unimportant and does not affect normal operation.

Further reading

This section lists publications by ARM Limited.

ARM periodically provides updates and corrections to its documentation. See <http://www.arm.com> for current errata sheets and addenda.

See also the ARM Frequently Asked Questions list at:
<http://www.arm.com/DevSupp/Sales+Support/faq.html>

ARM publications

This document contains information that is specific to the ARM PrimeCell SMC. Refer to the following document for other relevant information:

- *AMBA Specification Rev 2.0* (ARM IHI 0011).

Feedback

ARM Limited welcomes feedback both on the ARM PrimeCell SMC, and on the documentation.

Feedback on the ARM PrimeCell SMC

If you have any comments or suggestions about this product, please contact your supplier giving:

- the product name
- a concise explanation of your comments.

Feedback on this document

If you have any comments on about this document, please send email to errata@arm.com giving:

- the document title
- the document number
- the page number(s) to which your comments refer
- a concise explanation of your comments.

General suggestions for additions and improvements are also welcome.

Chapter 1

Introduction

This chapter introduces the ARM PrimeCell Static Memory Controller (PL092) and contains the following section:

- *About the ARM PrimeCell Static Memory Controller (PL092)* on page 1-2.

1.1 About the ARM PrimeCell Static Memory Controller (PL092)

The PrimeCell *Static Memory Controller* (SMC) is an *Advanced Microcontroller Bus Architecture* (AMBA) compliant System-on-Chip peripheral that is developed, tested, and licensed by ARM.

The PrimeCell SMC is an AMBA slave module, and connects to the *Advanced High-performance Bus* (AHB). It is a reusable soft-IP block that has been developed with the prime aim of reducing time-to-market for ASIC development. It contains an *AHB Test Interface Controller* (TIC) AMBA master block that can be used to test the system using externally applied TIC vectors.

A carefully chosen set of design rules has been followed to allow faster integration in most ASIC design flows using standard synthesis and test tools. The implementation can easily be customized to suit specific customer requirements. The AMBA TIC is included to reduce the design time needed to construct a PrimeCell SMC based system that allows testing through TIC vectors.

For further information on AMBA refer to the *AMBA Specification (Rev 2.0)*.

1.1.1 Features of the PrimeCell SMC

The PrimeCell SMC macro block offers the following features:

- soft macrocell available in both VHDL and Verilog
- fully scan insertable design
- functional verification using ARM BusTalk functional test environment
- compatibility with AMBA AHB on-chip bus systems.

The PrimeCell SMC supports:

- static memory-mapped devices including RAM, ROM, flash, and burst ROM
- asynchronous page mode read operation in nonclocked memory subsystems
- asynchronous burst mode read access from burst mode ROM and flash devices
- 8, 16, and 32-bit wide external memory data paths
- little-endian and big-endian memory architectures
- AHB burst transfers
- independent configuration for up to eight memory banks, each up to 64MB
- programmable wait states (up to 31)
- programmable bus turnaround cycles (up to 15)
- programmable output enable and write enable delays (up to 15)
- write enable and byte lane select outputs for use with 32, 16, or 8-bit SRAM devices
- independent byte lane control for each memory bank

- external asynchronous wait control
- configurable size at reset for boot memory bank using external control pins
- system testing using externally applied TIC vectors through built-in TIC AMBA master block
- access port allows a future device access to external bus.

1.1.2 Programmable parameters

The following key parameters are programmable for each memory bank:

- external memory width, 8, 16, or 32-bit
- burst mode operation
- write protection
- chip select polarity
- external wait control enable
- external wait polarity
- write WAIT states for static RAM devices
- read WAIT states for static RAM and ROM devices
- initial and subsequent burst read WAIT state for burst ROM devices
- read byte lane enable control
- bus turn-around (idle) cycles
- output enable and write enable output delays.

Figure 1-1 shows a diagram of a typical AHB-based microcontroller system, with an optional additional asynchronous memory controller. The PrimeCell SMC has two AHB connections, the slave interface for the SMC and the master interface for the TIC.

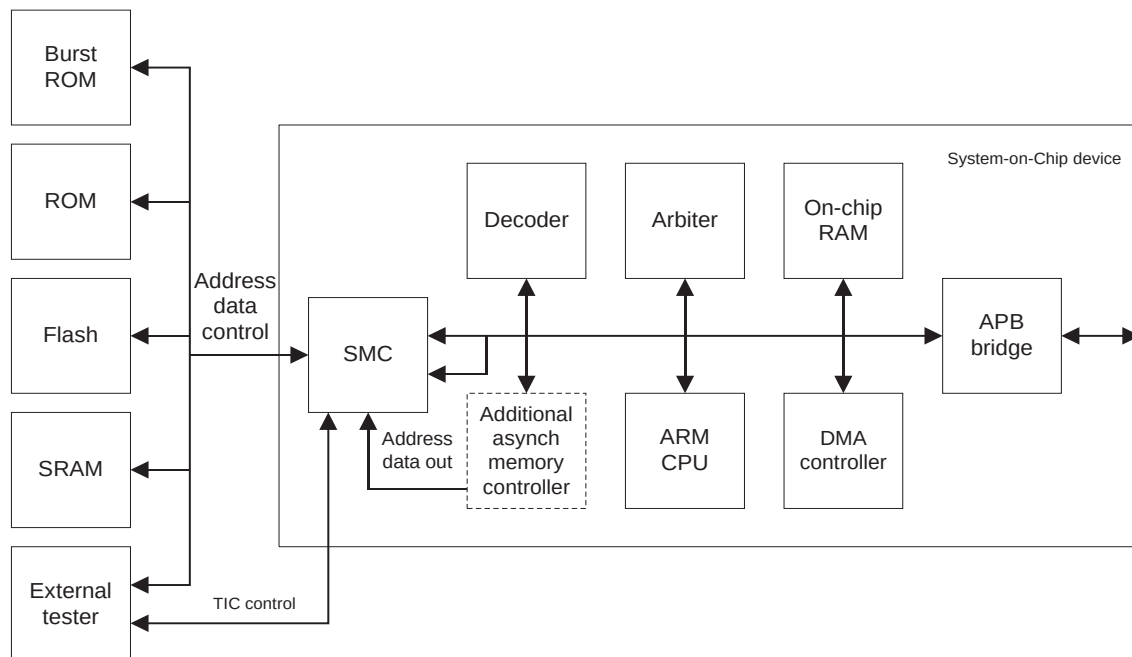


Figure 1-1 Typical AMBA AHB-based microcontroller system

The PrimeCell SMC also supports the connection of a synchronous AHB memory controller, such as an SDRAM controller. This is shown in Figure 1-2.

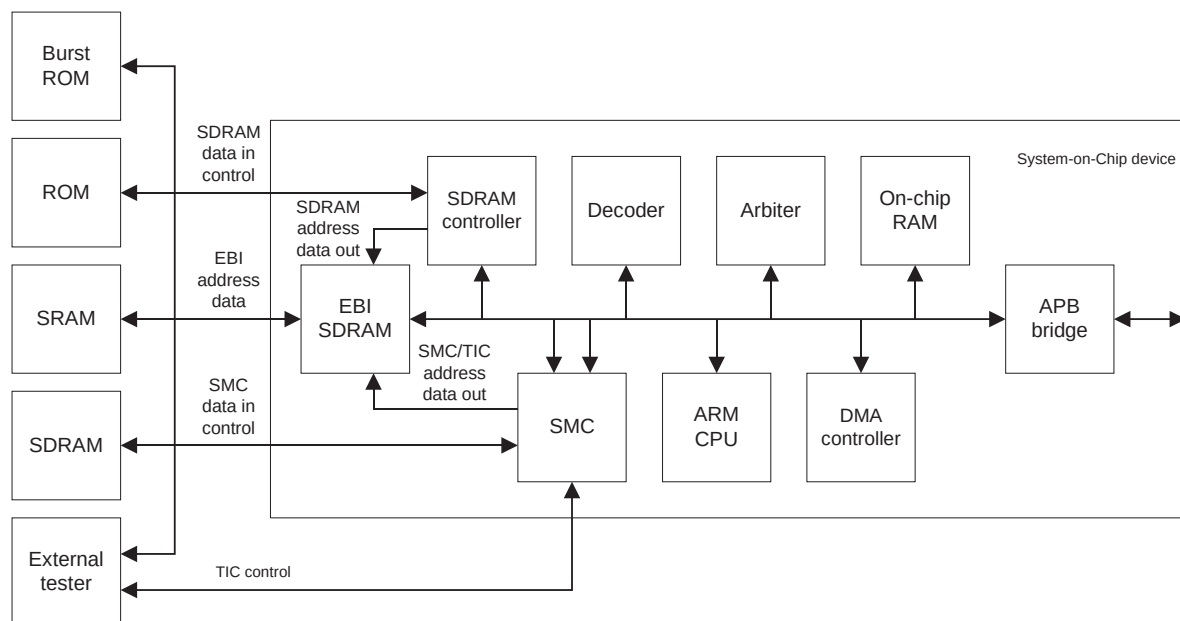


Figure 1-2 Typical AMBA AHB-based microcontroller system using a SMC and a SDRAM controller

Figure 1-3 shows the input and output connections for the PrimeCell SMC.

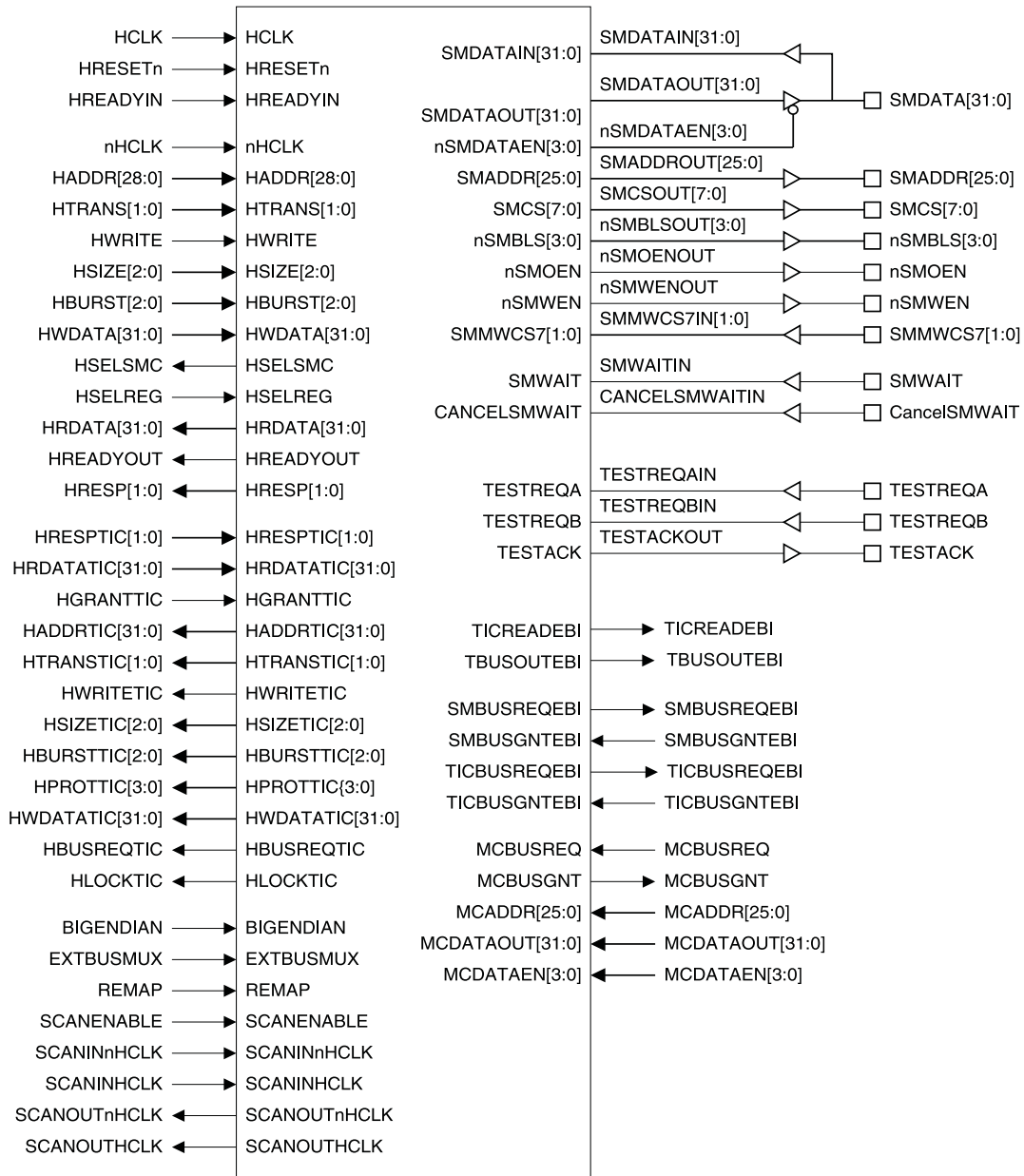


Figure 1-3 PrimeCell SMC input and output connections

Figure 1-4 shows the signal connections of an additional asynchronous memory controller to the PrimeCell SMC. Depending on the bus request and grant protocol supported by the additional memory controller, a simple wrapper might be required to allow it to operate with the protocol implemented in the PrimeCell SMC.

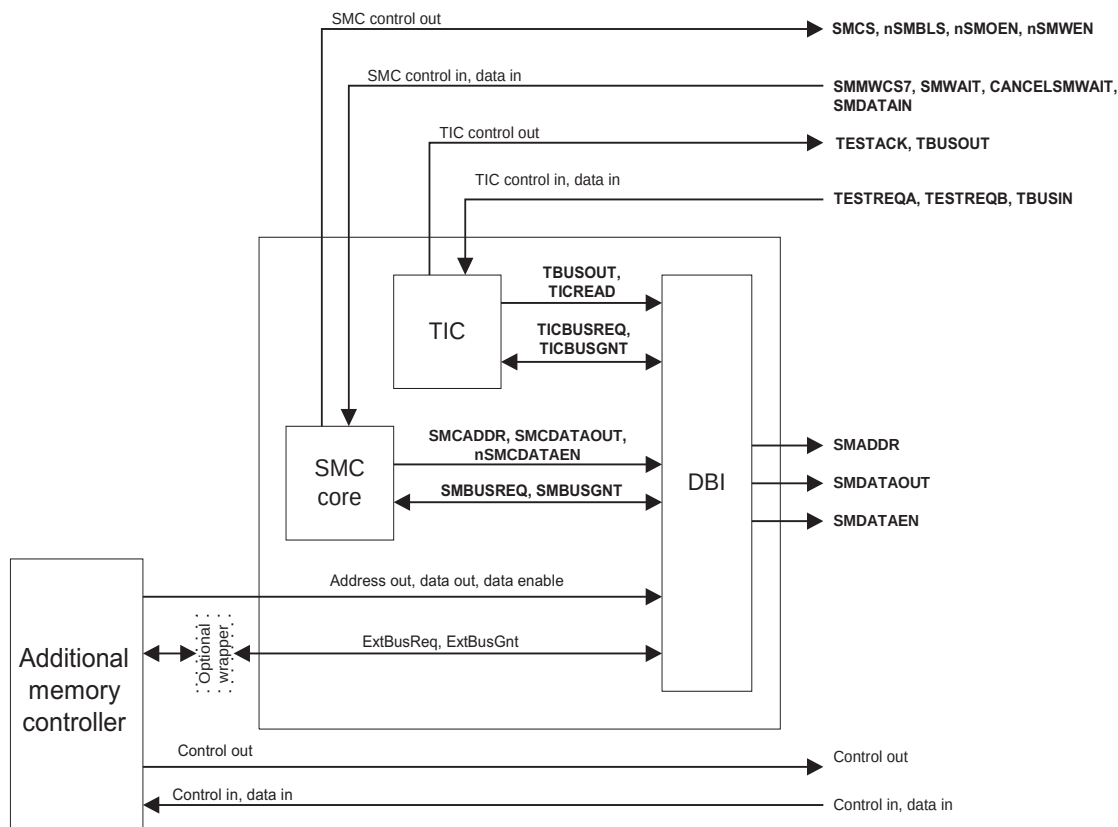


Figure 1-4 PrimeCell SMC signal connections when used with an additional asynchronous memory controller

Chapter 2

Functional Overview

This chapter describes the ARM PrimeCell Static Memory Controller (PL092) operation. It contains the following sections:

- *ARM PrimeCell SMC overview* on page 2-2
- *PrimeCell SMC operation* on page 2-6
- *Memory shadowing* on page 2-46
- *Test interface controller operation* on page 2-48
- *Data bus interface operation* on page 2-58
- *Using the SMC with an external bus multiplexer or SDRAM controller* on page 2-60.

2.1 ARM PrimeCell SMC overview

The PrimeCell *Static Memory Controller* (SMC) is an AMBA AHB slave module that can be used to provide an interface between an AMBA AHB system bus and external (off-chip) memory devices.

The PrimeCell SMC provides support for up to eight independently configurable memory banks simultaneously. Each memory bank is capable of supporting:

- SRAM
- ROM
- flash EPROM
- burst ROM memory.

Each memory bank can be configured to use either 8, 16, or 32-bit external memory data paths. The PrimeCell SMC can be configured to support either little-endian or big-endian operation.

The PrimeCell SMC memory banks can be configured to support:

- nonburst read and write accesses to high-speed CMOS Static RAM, for example Samsung KM681002A and K6R1016C1C, and Intel 28F800C3
- nonburst write accesses, nonburst read accesses, and asynchronous page mode read accesses to fast-boot block flash memory, for example Intel 28F800F3 and 28F128J3A
- asynchronous page mode read accesses to ROM, for example Samsung K3P5V(U)2000D-SC and K3P6C2000B-SC.

Support is provided for connecting an additional AHB asynchronous memory controller to external memory, to pass the address, data and data enable lines to the external bus. A simple request and grant protocol is used to control the current driver of the external bus. Use of a synchronous memory controller is also supported, but this requires the use of a suitable external bus multiplexer. See *Using the SMC with an external bus multiplexer or SDRAM controller* on page 2-60 for more details.

A block diagram of the PrimeCell SMC is shown in Figure 2-1.

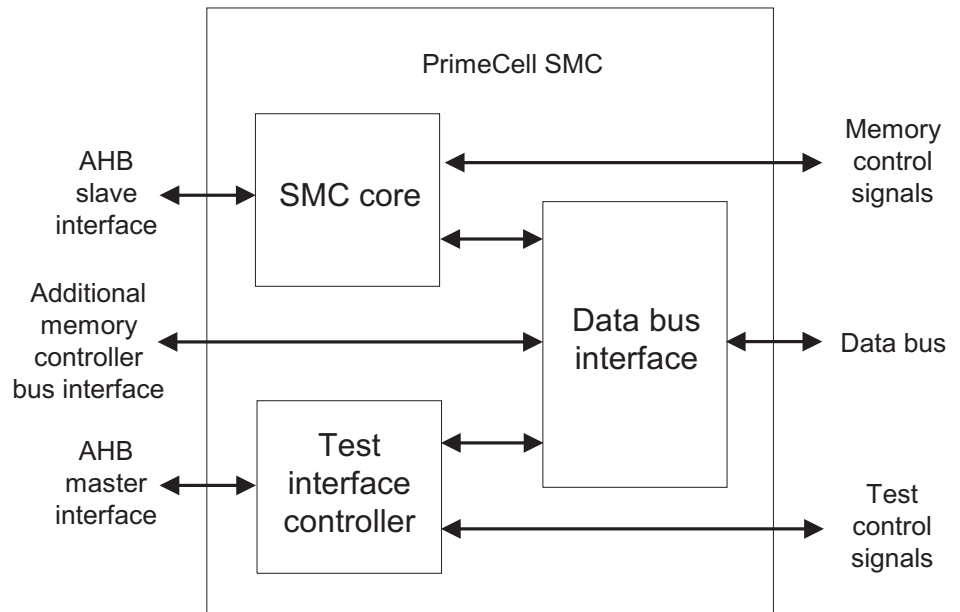


Figure 2-1 PrimeCell SMC block diagram

The three main sub-blocks in the PrimeCell SMC design are:

SMC core Performs read and write accesses to external memory through the AMBA AHB slave interface.

Test interface controller The TIC is used during testing to read external test vectors and apply them to the system through the AMBA AHB master interface.

Data bus interface The data bus interface selects either the memory controller, the TIC, or the additional memory controller as the current user of the external data bus.

2.1.1 SMC core

The SMC core performs read and write accesses to external memory through the AMBA AHB slave interface. Figure 2-2 shows a block diagram of the SMC core.

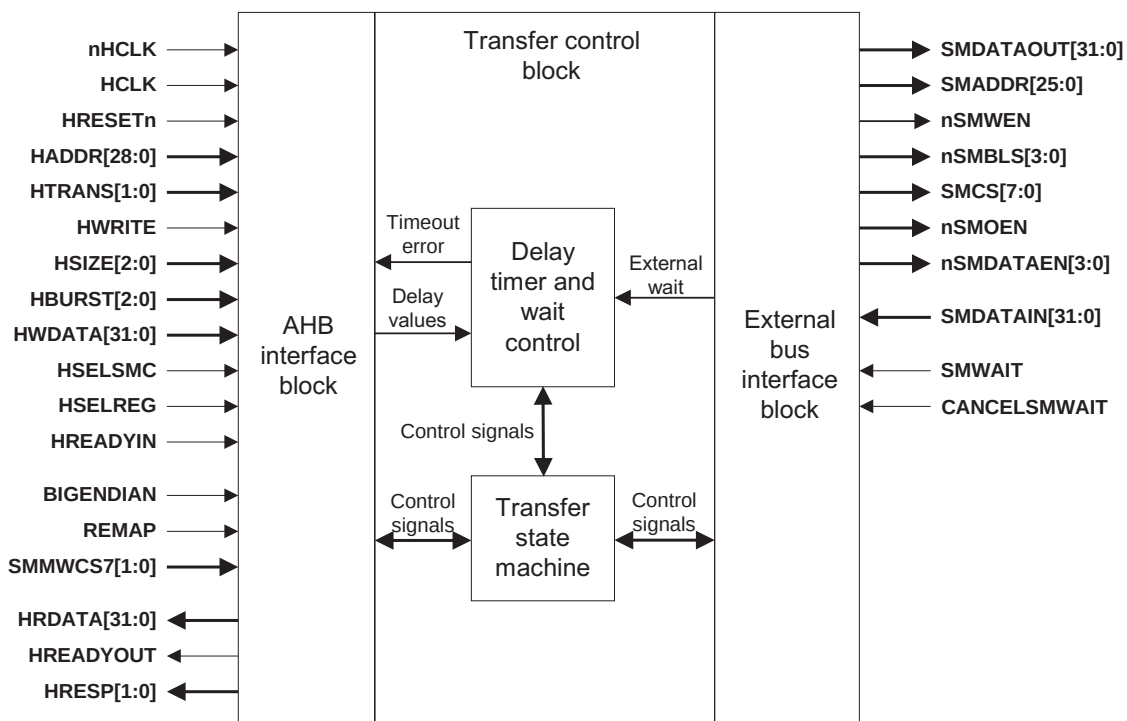


Figure 2-2 SMC core block diagram

The three main blocks of the SMC core are:

- *AMBA AHB interface* on page 2-5
- *Transfer control* on page 2-5
- *External bus interface* on page 2-5.

AMBA AHB interface

The AMBA AHB interface block provides the interface to the AMBA AHB bus. It contains all of the control and status registers for the external memory banks, and the PrimeCell and Peripheral Identification registers.

The AHB control signals are sampled and passed to the transfer control block, and the control signals received are used to generate the AHB response and read data outputs.

The read/write control register values are accessed through the AHB, and their values are passed to the rest of the peripheral. The read only status register values are generated from control signals within the transfer control block.

Transfer control

The transfer control block contains two main sub-blocks:

Transfer state machine

This block is used to control the transfer currently being performed.

Delay timer and wait control

This block is used to control the operation of reads, writes and external waits when a delay is required, and to control the externally waited transfers.

External bus interface

The external bus interface block provides the interface to the external bus. It contains the logic to generate the external control signals, and the 32-bit read/write buffer used to store the data value during transfers. It is used to:

- drive the chip selects for the memory banks
- drive the write enables for the memory banks
- drive the output enable for the memory banks
- drive the external databus enable
- generate read enables for read data from the memory to AMBA AHB interface
- control the data sequencing for memory read and writes for different sizes of AHB transfer and memory width.

2.2 PrimeCell SMC operation

This section describes the operation of the PrimeCell SMC, including the timing of standard transfers for different memory types and externally waited transfers, and example configurations for different memory device and bank sizes. The functions of PrimeCell SMC are described under the following headings:

- *Memory bank select* on page 2-6
- *Access sequencing and memory width* on page 2-7
- *Wait state generation* on page 2-8
- *Write protection* on page 2-8
- *Static memory read control* on page 2-9
- *Static memory write control* on page 2-19
- *Byte lane control* on page 2-33.

2.2.1 Memory bank select

Eight independently configurable memory banks are supported, with a separate chip select output for each bank. The chip select lines **SMCS[7:0]** for all banks are configurable to be either active HIGH or active LOW (default). The memory bank selection is controlled by the AMBA AHB address lines **HADDR[28:26]**, as shown in Table 2-1, where all **SMCS** are shown as active HIGH.

Table 2-1 Static memory bank select coding

HADDR [28:26]	SMCS[7:0]	Memory bank
000	00000001	Bank 0
001	00000010	Bank 1
010	00000100	Bank 2
011	00001000	Bank 3
100	00010000	Bank 4
101	00100000	Bank 5
110	01000000	Bank 6
111	10000000	Bank 7

The base address of the external memory banks and the base address of the PrimeCell SMC memory bank registers are defined in the AMBA AHB address decoder, that is used to generate the AHB slave select signals **HSELSMC** and **HSELREG**.

If the default base address of the external memory banks begins at 0x00000000, then the memory banks occupy address space up to 0x1FFFFFFF (8 x 64MB).

Table 2-2 shows the address mapping of **HADDR[31:0]** for external memory banks.

Table 2-2 Address mapping for external memory banks

31-29	28-26	25-0
Base address for PrimeCell SMC memory	Chip select address space for eight memory banks	64MB memory banks address space

Table 2-3 shows the address mapping of **HADDR[31:0]** for memory bank configuration registers.

Table 2-3 Address mapping for memory bank registers

31-29	28-12	11-2
Base address for PrimeCell AHB SMC registers	Unused	Memory bank register select

———— **Note** —————
HADDR[31:29] are not inputs to the SMC since decoding is performed in the AMBA address decoder.

2.2.2 Access sequencing and memory width

The data width of each external memory bank must be configured by programming the appropriate bank configuration register SMBCRx. When the external memory bus is narrower than the transfer initiated from the current AMBA bus master, the internal bus transfer takes several external bus transfers to complete. For example, in the case that Bank 0 is configured as 8-bit wide memory and a 32-bit read is initiated, the AMBA AHB bus stalls while the PrimeCell SMC reads four consecutive bytes from the memory. During these accesses the data path is controlled (in the external memory data path logic) to demultiplex the four bytes into one 32-bit word on the AMBA AHB bus.

The access sequencing supports both little-endian and big-endian operation as defined by the dedicated **BIGENDIAN** input signal to the PrimeCell SMC.

2.2.3 Wait state generation

Each bank of the PrimeCell SMC must be configured for external transfer wait states in read and write accesses. This is achieved by programming the appropriate fields of the bank control registers SMBIDCYRx, SMBWST1Rx, and SMBWST2Rx. The number of cycles in which an AMBA transfer completes is controlled by three other factors:

- access width
- external memory width
- external wait input.

Each bank of the PrimeCell AHB SMC has a programmable enable for the external wait (WaitEn), and a programmable polarity setting (WaitPol), allowing full configuration of the external wait for each bank.

The WST1 wait state field can be programmed to select up to 31 wait states for read memory accesses to SRAM and ROM, or the initial burst read access to burst ROM.

The WST2 wait state field can be programmed to select up to 31 wait states for write access to SRAM or burst mode reads from burst ROM devices. For example, the configuration for an access to a burst ROM with a 120ns initial access time followed by a 60ns burst access time, using a 100MHz system clock would be 12 wait states for the first access and 6 for the subsequent accesses.

2.2.4 Write protection

Each memory bank can be configured for write protection. Normally SRAM is unprotected and ROM devices must be write protected, but the WP field in the bank configuration registers SMCBCRx can be set to write protect SRAM as well as ROM devices.

If a write access is made to a write protected memory bank, an error is indicated by the **HRESP[1:0]** signals and the WriteProtErr bit of the status register is asserted. If a write access is made to a memory bank containing ROM devices and the bank is not write protected, there is no error indication returned.

2.2.5 Static memory read control

The static memory read controls are described in the following sections:

- *Output enable programmable delay* on page 2-9
- *ROM, SRAM, and flash* on page 2-9
- *Burst ROM* on page 2-16
- *Burst flash* on page 2-18.

Output enable programmable delay

The delay between the assertion of the chip select and the output enable is programmable from 0 to 15 cycles using the WSTOEN bits of the bank control registers. This delay is used to reduce the power consumption for memories that are not able to provide valid output data immediately after the chip select is asserted. If the output of the device is enabled before the final read data value is ready, the device drives out two different values, one unknown value, followed by the valid read data. This consumes more power than just driving out the final read data value. The output enable is always deasserted at the same time as the chip select, at the end of the transfer.

Note

The WSTOEN programmed value must be equal to, or less than the WST1 programmed value, as the access is timed by the wait states and not by the WSTOEN value.

In the External Wait enabled mode, the timing of the transfer (controlled by **SMWAIT**) is not known, so **SMOEN** is asserted along with **SMCS**.

ROM, SRAM, and flash

The PrimeCell SMC uses the same read timing control for ROM, SRAM, and flash devices. Each read starts with the assertion of the appropriate memory bank chip select signals **SMCS[x]** and memory address **SMADDR[25:0]**. The read access time is determined by the number of wait states programmed for the WST1 field of the bank control register SMBWST1Rx. The IDCY field in the idle cycle control register SMBIDCYRx determines the number of bus turnaround wait states added between external read and write transfers.

Figure 2-3 shows an external memory read transfer with the minimum zero wait states (WST1 = 0), and the minimum zero output enable delay states (WSTOEN=0). A minimum of two AHB wait states are inserted during all single read transfers.

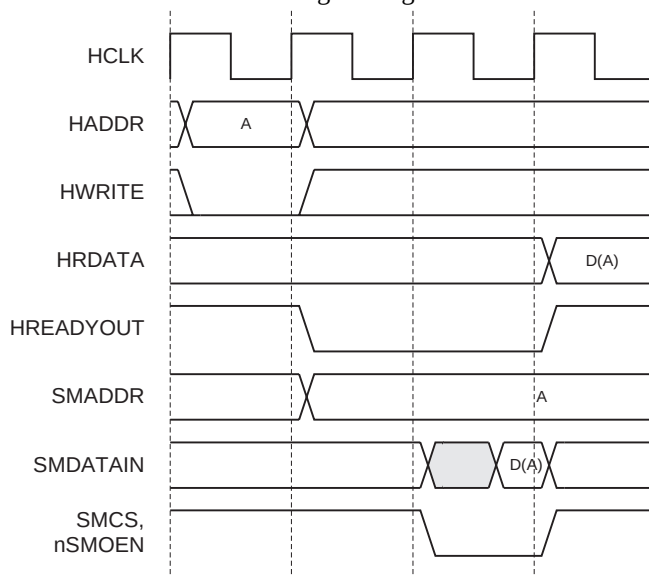


Figure 2-3 External memory zero wait state read timing diagram

Figure 2-4 shows an external memory read transfer with two wait states ($WST1 = 2$), and the minimum zero output enable delay states ($WSTOEN=0$). Four AHB wait states are inserted during the transfer, two for the standard read, and an additional two due to the programmed wait states added.

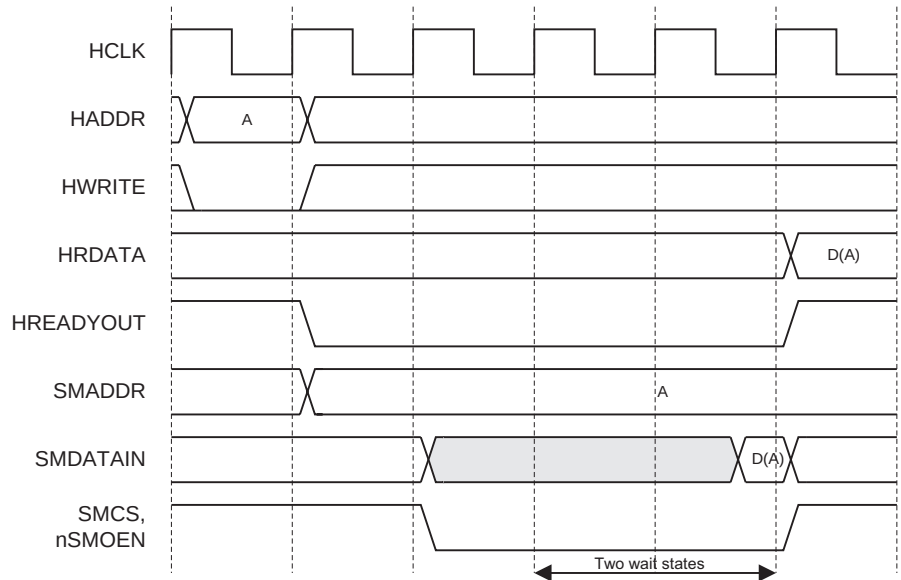


Figure 2-4 External memory two wait state read timing diagram

Figure 2-5 shows an external memory read transfer with two output enable delay states (WSTOEN = 2) and two wait states (WST1 = 2). Four AHB wait states are inserted during the transfer, two for the standard read, and an additional two due to the output enable delay states added.

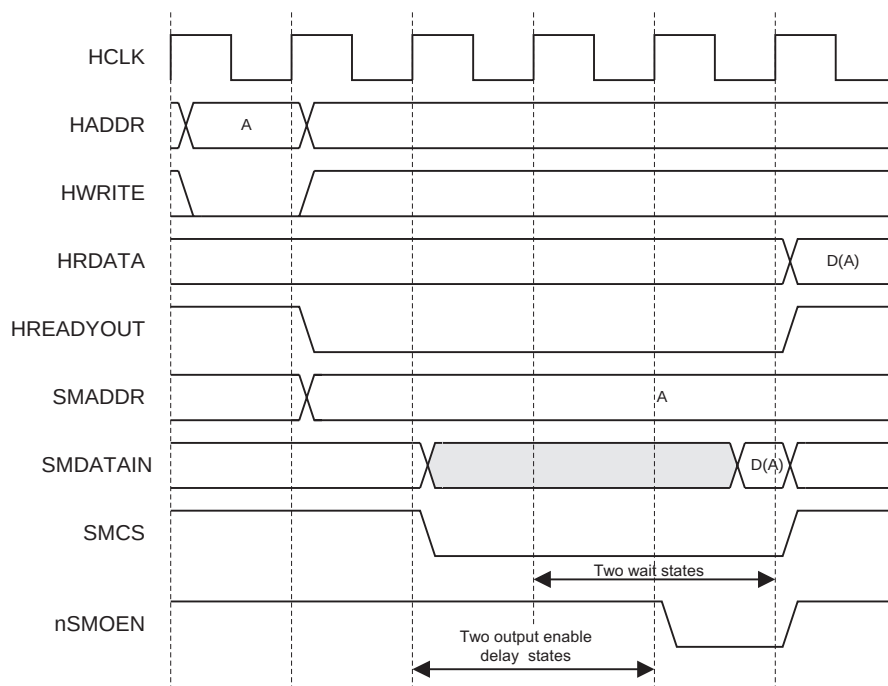


Figure 2-5 External memory two output enable delay and two wait state read timing diagram

Figure 2-6 shows an external memory read transfer with the minimum zero wait states where the SMC does not have control of the bus and must request for it. In this example nothing else is requesting the bus, so the SMC is granted straight away, showing the minimum timing when the bus is requested.

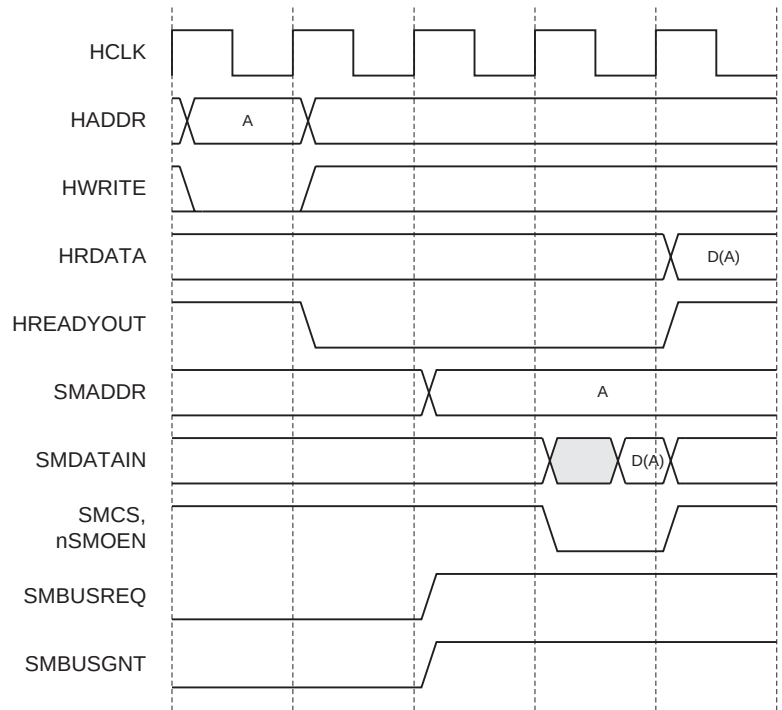


Figure 2-6 External memory zero wait state read when not granted the bus timing diagram

Figure 2-7 shows external memory read transfers with zero wait states (WST1 = 0). These might be nonsequential transfers, or sequential transfers of unspecified burst length. All transfers are treated as separate reads, so have the minimum of two AHB wait states added.

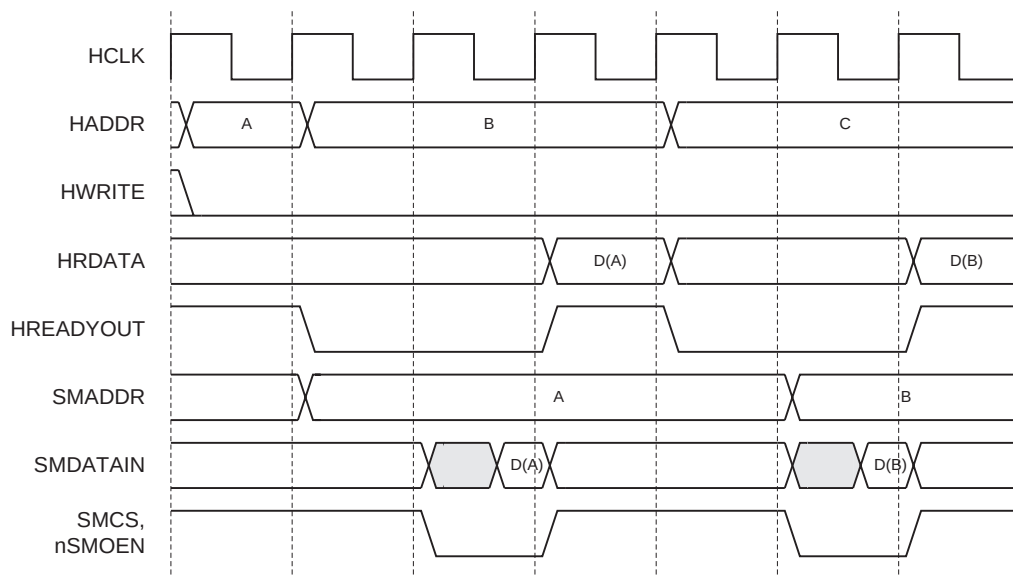


Figure 2-7 External memory three zero wait state read timing diagram

Figure 2-8 shows a burst of zero wait state reads with the length specified. As the length of the burst is known, it is possible to hold the chip select asserted during the whole burst, and generate the external transfers before the current AHB transfer has completed. Therefore, the first read has two AHB wait states added, and the three following sequential reads have zero AHB wait states added due to the automatic generation of the external transfers.

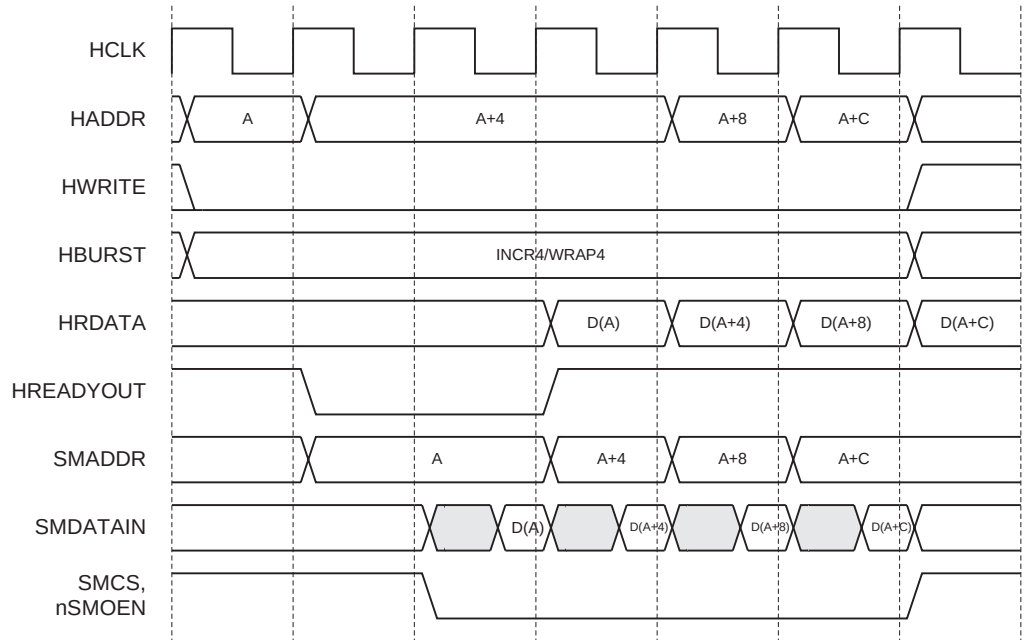


Figure 2-8 External memory zero wait fixed length read timing diagram

Figure 2-9 shows a burst of two wait state reads with the length specified. The WST1 value is used for all transfers in the burst, with the first read having four AHB wait states inserted, and all sequential transfers having two AHB wait states.

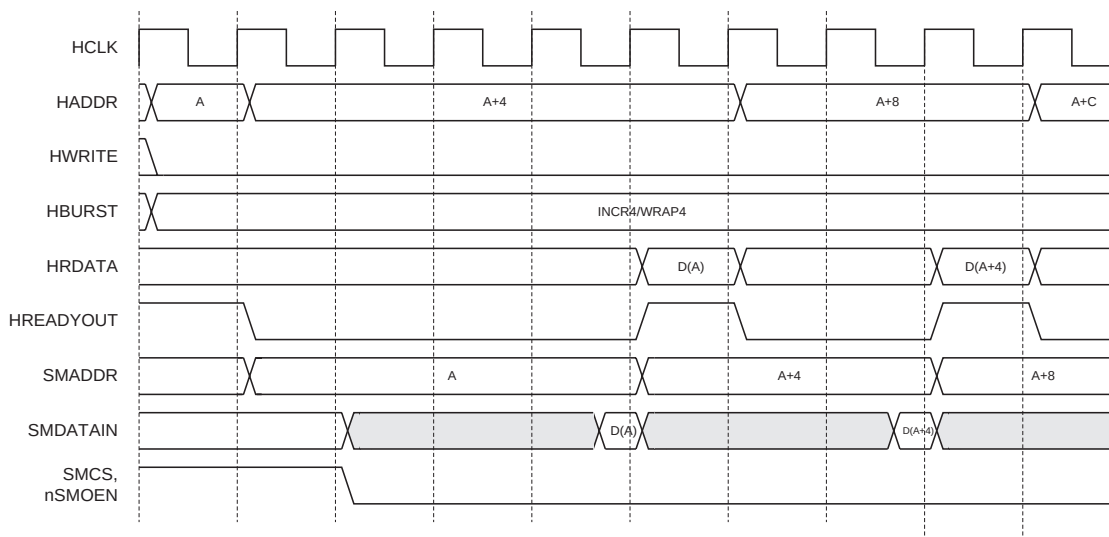


Figure 2-9 External memory two wait states fixed length burst read timing diagram

Burst ROM

The PrimeCell SMC supports sequential access burst reads to a maximum of four consecutive locations in 8, 16 or 32-bit memories. This feature supports burst mode ROM devices and increases the bandwidth by using a reduced (configurable) access time for the sequential reads (WST2) following the first read (WST1). The chip select and output enable lines are held during the burst, and only the address changes between subsequent accesses. At the end of the burst the chip select and output enable lines are deasserted together.

Note

Bursts cannot cross quad boundaries, which are:

SMADDR[1:0] = 11 for 8-bit transfers

SMADDR[2:1] = 11 for 16-bit transfers

SMADDR[3:2] = 11 for 32-bit transfers.

They are split up so that the first transfer after the boundary uses the slow read (WST1) timing.

For example, a four byte transfer starting at address **SMADDR[1:0]** = 01 performs a slow read from address 01, two fast reads from 10 and 11, and then a final slow read from address 00 to finish the burst.

Figure 2-10 shows an external memory burst read transfer with two initial wait states, and one sequential wait state. The first read has four AHB wait states inserted, and all additional sequential transfers have only one AHB wait state. This gives increased performance over the equivalent nonburst ROM timing shown in Figure 2-9 on page 2-16.

Note

External burst transfers are always split up into bursts of maximum four transfers, with the first read using the slow timing and the subsequent three reads using the fast timing, due to the four transfer burst limit of burst ROM devices. This four transfer limit is only applied to the external transfers. An AHB burst with size greater than the external memory is split up into bursts of four external transfers, taking into account any quad boundary connections.

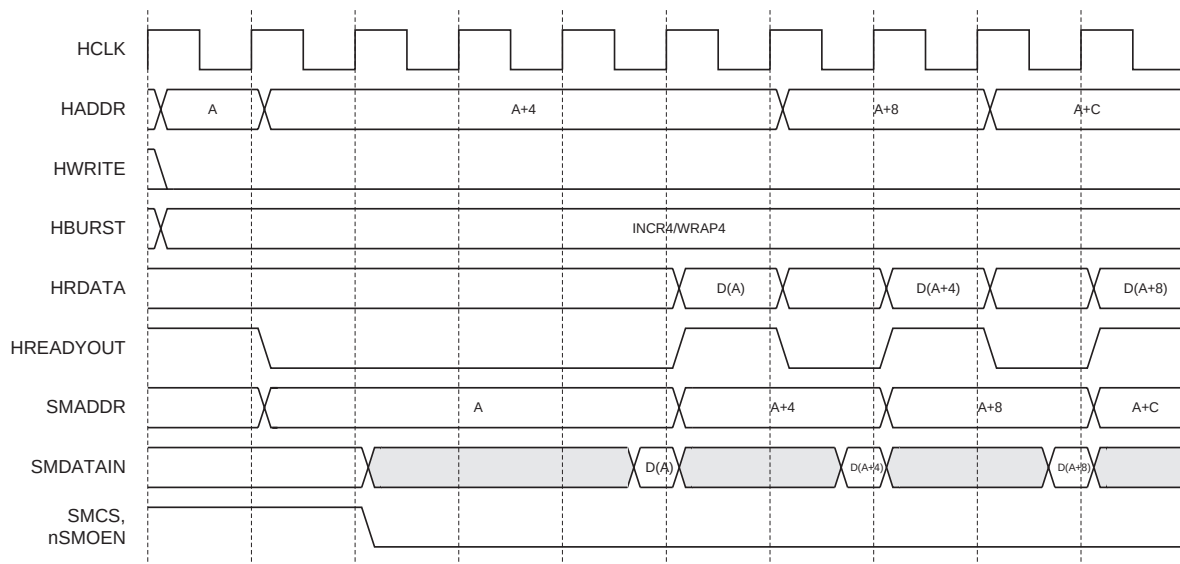


Figure 2-10 External burst ROM WST1 = 2 and WST2 = 1 fixed length burst read timing diagram

Figure 2-11 shows a 32-bit read from an 8-bit burst ROM device, causing four burst reads to be performed. A total of five AHB wait states are added during this transfer, two for the first external read, and then one for each of the subsequent reads.

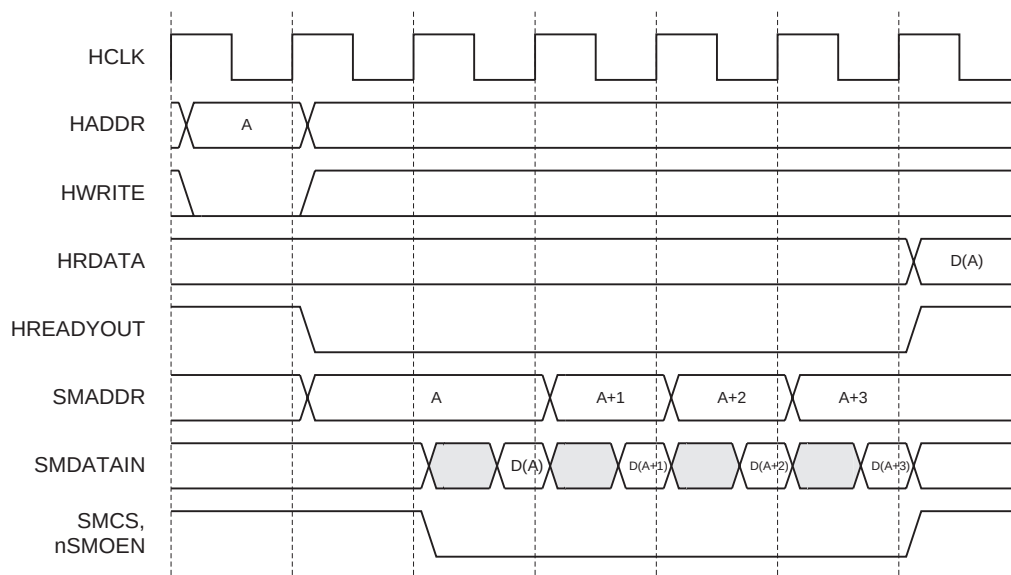


Figure 2-11 External memory 32-bit burst read from 8-bit memory timing diagram

Burst flash

The PrimeCell SMC supports sequential access burst reads from burst flash devices, of the same types as for burst ROM. Due to the sharing of the WST2 register between write transfers and burst read transfers, it is only possible to have one setting at a time for burst flash, either the write delay or the burst read delay. This means that for a write transfer the WST2 register must be programmed with the write delay value, and for a burst read transfer the WRT2 register must be programmed with the burst access delay value.

2.2.6 Static memory write control

Write timing is described in the following sections:

- *Write enable programmable delay* on page 2-19
- *SRAM* on page 2-19
- *Flash memory* on page 2-25.

Write enable programmable delay

The delay between the assertion of the chip select and the write enable is programmable from 0 to 15 cycles using the WSTWEN bits of the bank control registers. This delay is used to reduce the power consumption for memories. The write enable is asserted on the falling edge of **HCLK** (rising edge of **nHCLK**) after the assertion of the chip select for zero wait states. The write enable is always deasserted half a cycle before the chip select, at the end of the transfer. **nSMBLS** has the same timing as **nSMWEN** for writes to 8-bit devices that use the byte lane selects instead of the write enables.

———— Note —————

The WSTWEN programmed value must be equal to, or less than the WST2 programmed value, as the access is timed by the wait states and not by the WSTWEN value.

In the External Wait enabled mode, the timing of the transfer (controlled by **SMWAIT**) is not known, so **nSMWEN** is asserted immediately after **SMCS**.

SRAM

Write timing for SRAM starts with assertion of the appropriate memory bank chip selects **SMCS[x]** and address signals **SMADDR[25:0]**. The write access time is determined by the number of wait states programmed for the WST2 field of the bank control register SMBWST2Rx. The IDCY field in the bank control register determines the number of bus turnaround wait states added between external read and write transfers.

Figure 2-12 shows a single external memory write transfer with minimum zero wait states ($WST2 = 0$), and the minimum zero write enable delay states ($WSTWEN=0$). No AHB wait states are added.

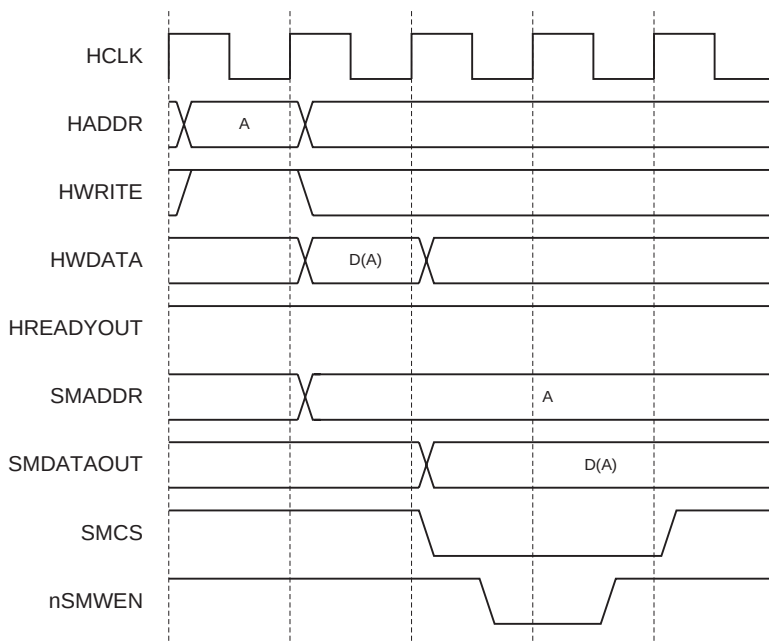


Figure 2-12 External memory zero wait state write timing diagram

Figure 2-13 shows a single external memory write transfer with two wait states (WST2 = 2), and the minimum zero write enable delay states (WSTWEN=0).

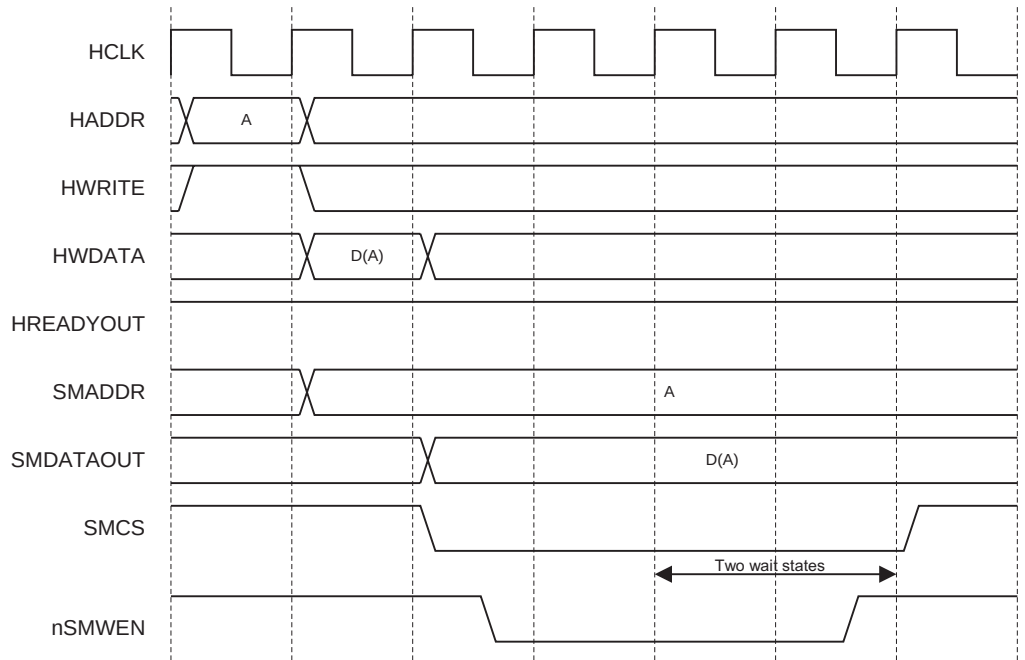


Figure 2-13 External memory two wait state write timing diagram

Figure 2-14 shows a single external memory write transfer with two write enable delay states (WSTWEN = 2) and two wait states (WST2 = 2). No AHB wait states are added.

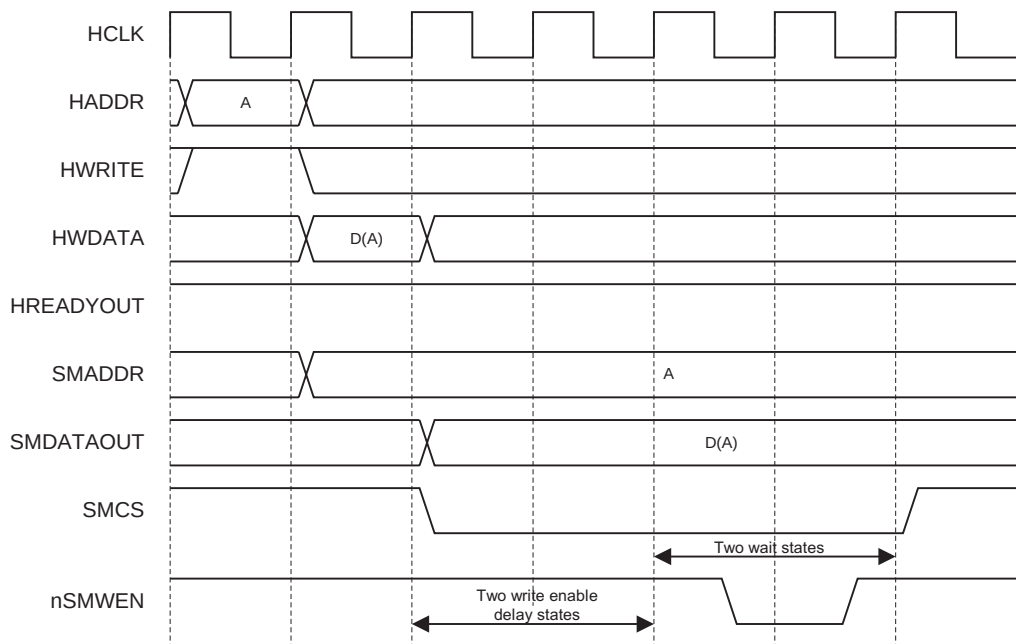


Figure 2-14 External memory two write enable delay and two wait state write timing diagram

Figure 2-15 shows a single external memory write transfer with minimum zero wait states where the SMC does not have control of the bus and must request for it. In this example, nothing else is requesting the bus, so the SMC is granted straight away, showing the minimum timing when the bus is requested.

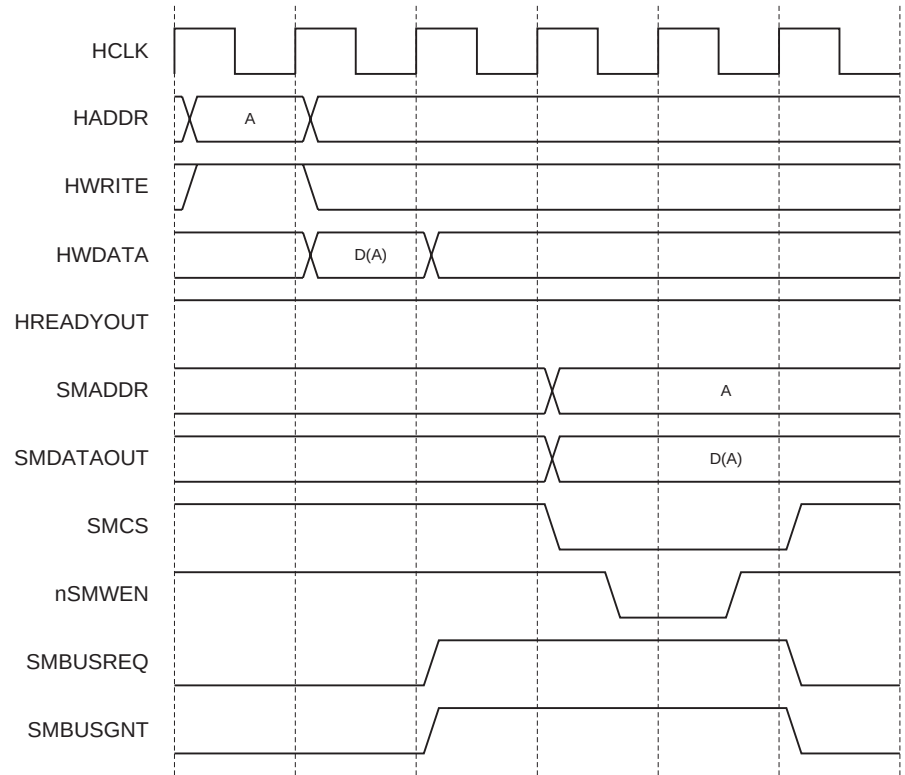


Figure 2-15 External memory zero wait state write when not granted the bus timing diagram

Figure 2-16 shows a single external memory write transfer with minimum zero wait states where the SMC does not have control of the bus and must request for it using an external synchronous bus multiplexer when a device such as an SDRAM controller is used. In this example nothing else is requesting the bus, so the SMC is granted straight away, showing the minimum timing when the bus is requested. See *Using the SMC with an external bus multiplexer or SDRAM controller* on page 2-60 for more details on the settings required for use with an SDRAM controller.

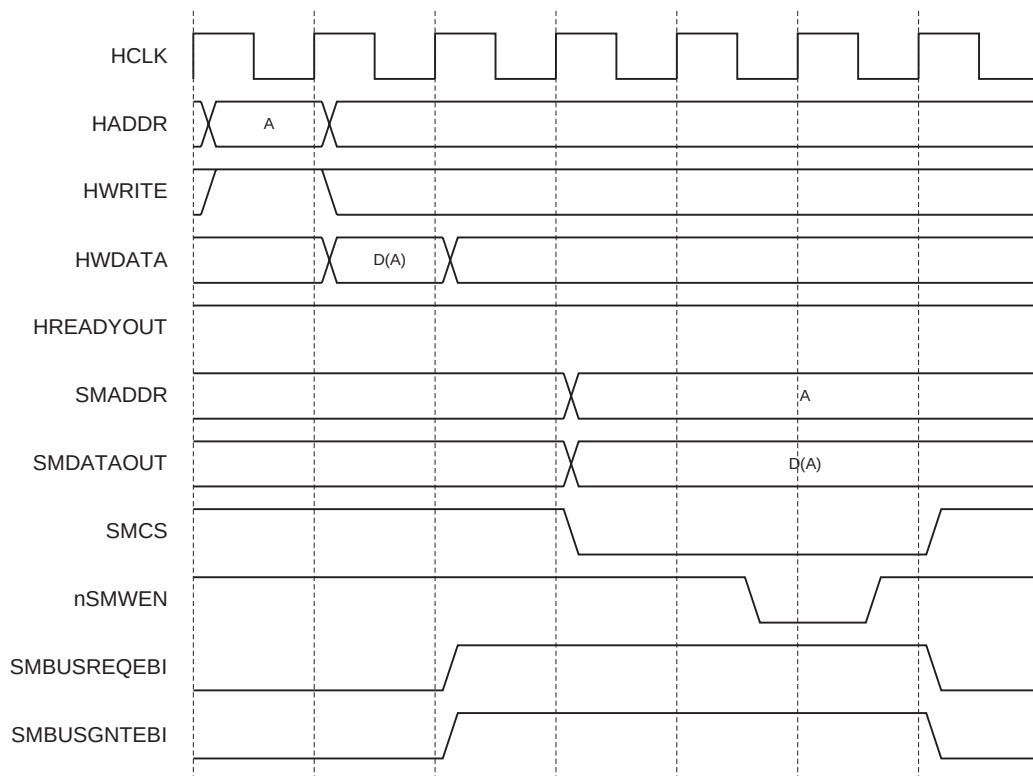


Figure 2-16 External memory zero wait state write when not granted the bus and using an external synchronous bus multiplexer timing diagram

Figure 2-17 shows two external memory write transfers with zero wait states ($WST2 = 0$). AHB wait states are added until the completion of the second write transfer. This is the timing of any sequence of write transfers:

- nonsequential to nonsequential
- or nonsequential to sequential with any value of **HBURST**.

The maximum speed of write transfers is controlled by the external timing of the write enable relative to the chip select. All external writes must take a minimum of two cycles to complete, the cycle that write enable is asserted, and the cycle that write enable is deasserted.

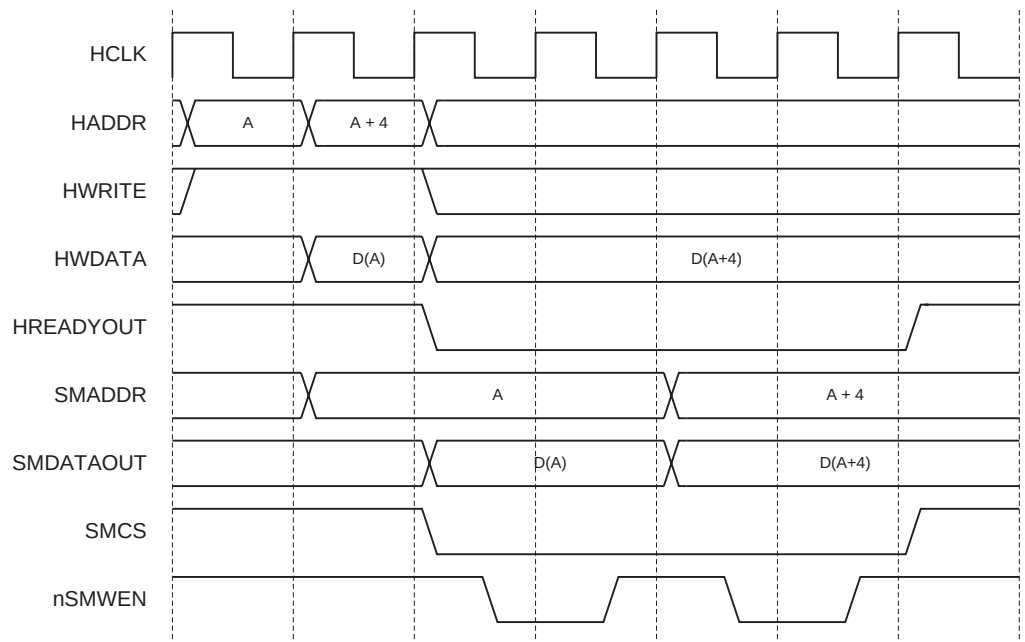


Figure 2-17 External memory two zero wait writes timing diagram

Flash memory

Write timing for flash memory devices is the same as for SRAM devices.

2.2.7 Bus turnaround

The PrimeCell AHB SMC can be configured for each memory bank to use external bus turnaround cycles between read and write memory accesses. The IDCY field can be programmed for up to 15 bus turnaround wait states. This is to avoid bus contention on the external memory data bus. Bus turnaround cycles are generated between external bus transfers as follows:

- read-to-read, to different memory banks
- read-to-write, to the same memory bank
- read-to-write, to different memory banks.

Figure 2-18 shows a zero wait read followed by a zero wait write with default turnaround between the transfers of two cycles due to the timing of the AHB transfers. Standard AHB wait states are added to the transfers, two for the read, and zero for the write.

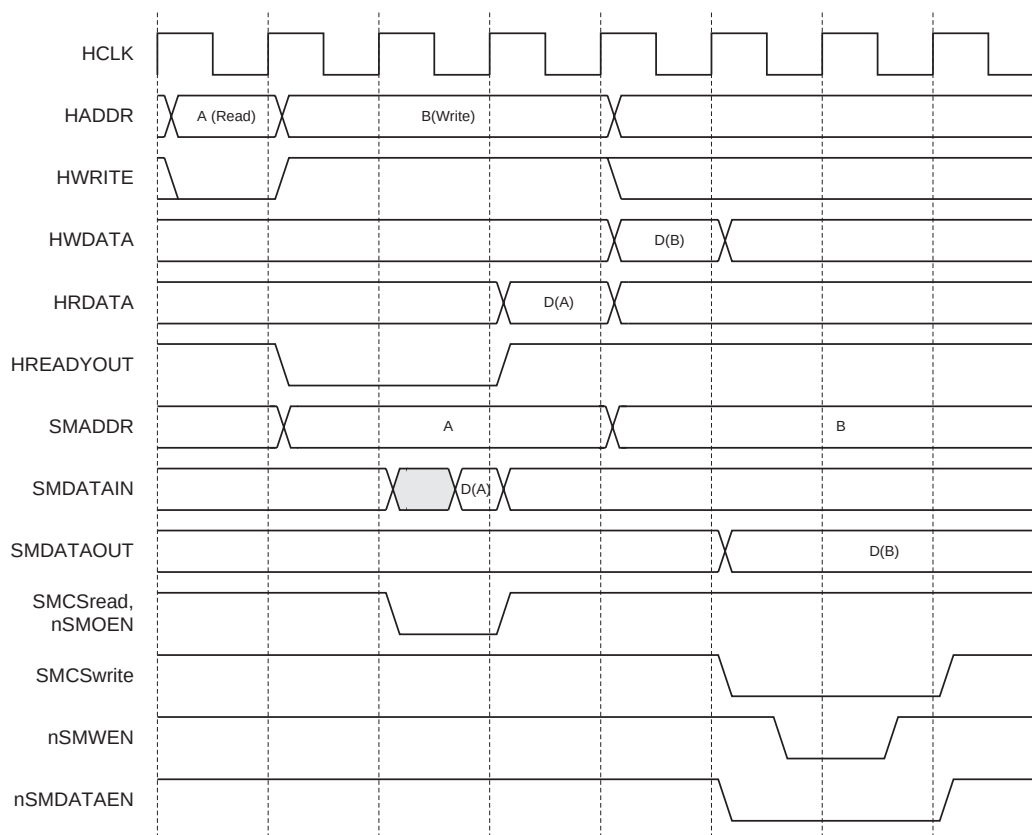


Figure 2-18 Read followed by write (both zero wait) with no turnaround

Figure 2-19 shows a zero wait write followed by a zero wait read with default turnaround between the transfers of one cycle. No AHB wait states are added to the write transfer, but four are added to the read, two to allow the write to complete before the read is started, and then the standard two for the read transfer.

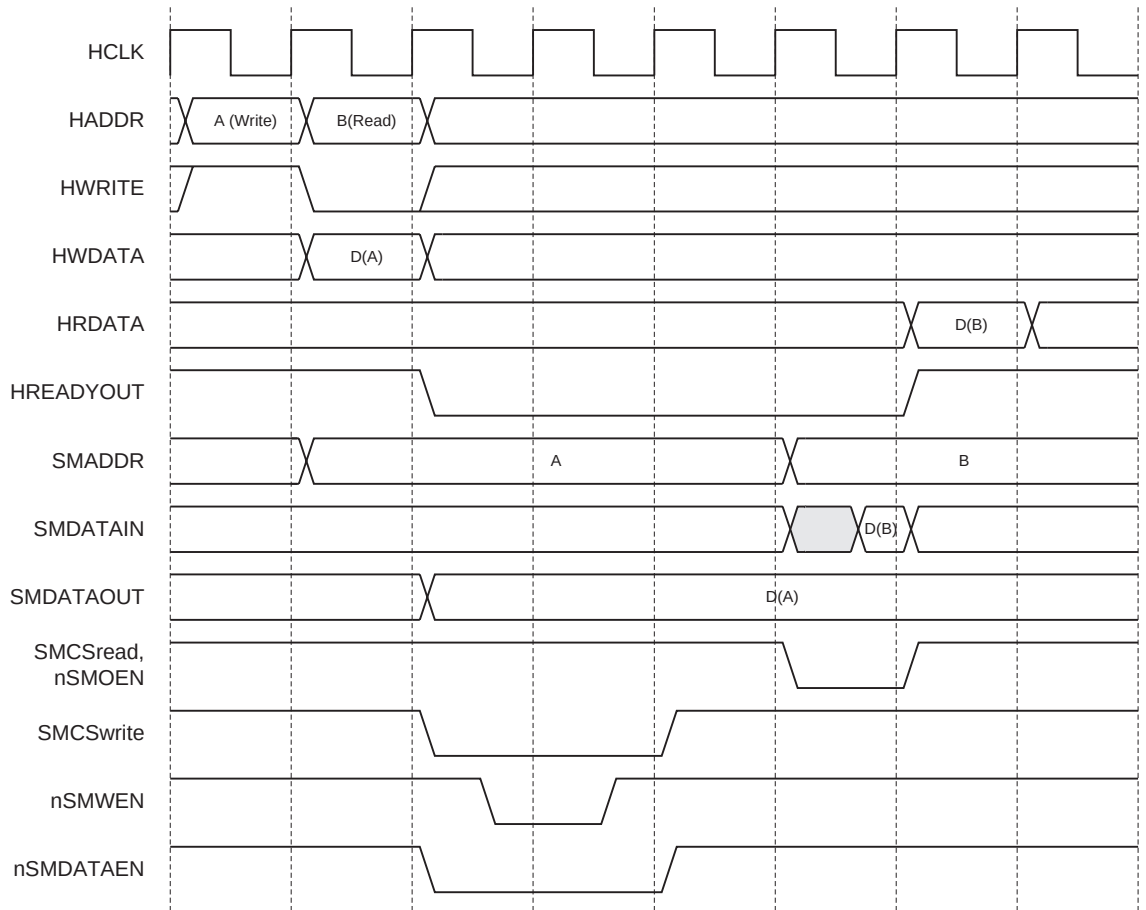


Figure 2-19 Write followed by read (both zero wait) with no turnaround

Figure 2-20 shows a zero wait read followed by two zero wait writes with two turnaround cycles added. The standard minimum of two AHB wait states are added to the read transfer, and none are added to the first write (as for any read-write transfer sequence). Two AHB wait states are added to the second write due to insertion of the two turnaround cycles that are only generated after the first write transfer is detected.

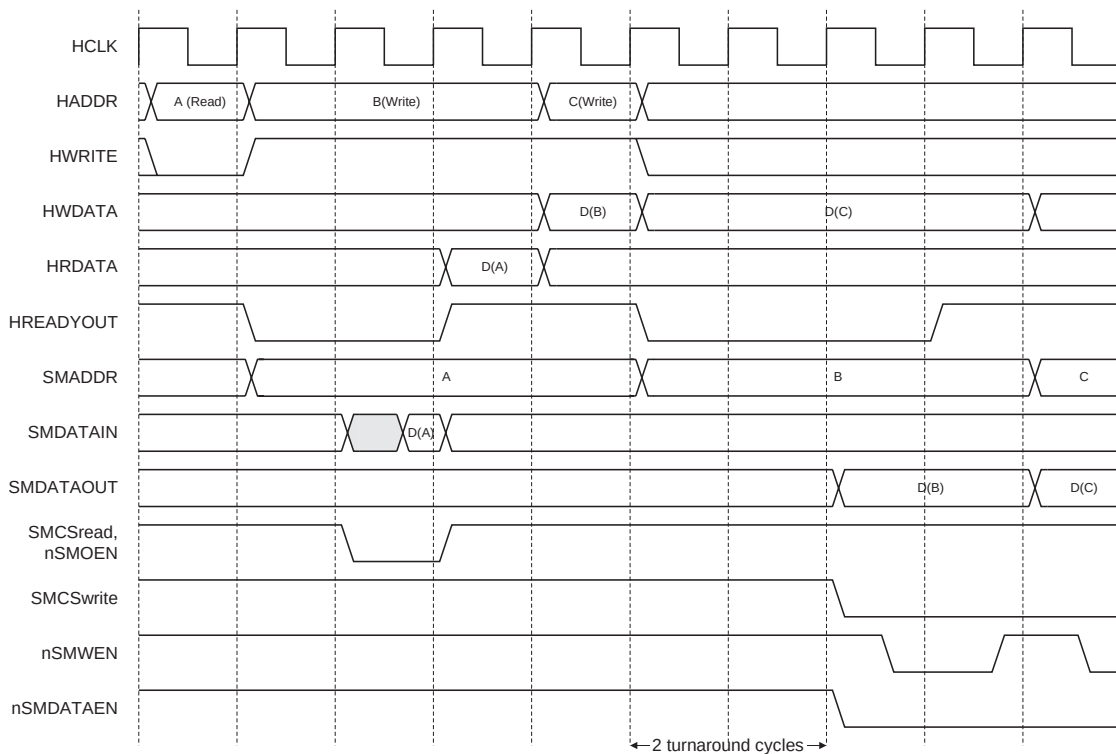


Figure 2-20 Read followed by two writes (all zero wait state) with two turnaround cycles

2.2.8 External wait control

The PrimeCell SMC supports the extension of the access cycle by an external device like a memory controller by using the **SMWAIT** input pin of the PrimeCell SMC. For this the WaitEn bit of the bank control registers SMBCRx must be programmed appropriately. The polarity of the external **SMWAIT** input is programmed through the WaitPol field of the SMBCRx register. If a problem occurs the active HIGH **CANCELSMWAIT** input allows externally waited transfers to be terminated early.

Note

As the external wait control inputs (**SMWAIT** and **CANCELSMWAIT**) are asynchronous inputs, they are synchronized before use. This gives all operations using external waits a two cycle delay due to the synchronization time.

When external wait mode is enabled, the SMC checks for assertion of the **SMWAIT** input, and waits the current transfer while **SMWAIT** stays asserted. The transaction completes once the **SMWAIT** line is deasserted (taking into account the two cycle synchronization delay).

If the external wait control mode is disabled, then the SMC ignores the **SMWAIT** input and the access time is generated normally according to the values programmed in the WST1 and WST2 registers.

SMWAIT assertion timing

In the wait enabled or external wait control mode, when the SMC is waiting for the **SMWAIT** assertion, it also starts counting down according to the values programmed in the wait state count field WST1 or WST2, that are used for read and write transfers respectively. You can use this feature to ensure that adequate time is available to the SMC to detect **SMWAIT** as there might be a delay before the external device asserts **SMWAIT**. If **SMWAIT** is not asserted during this time, the transfer is assumed to be zero wait.

SMWAIT deassertion timing

A waited transfer only ends when the **SMWAIT** input has been deasserted. The **CANCELSMWAIT** input is provided to allow the system to recover if the external device waits the transfer for longer than expected. A timer or watchdog must be used to control the assertion of the active HIGH **CANCELSMWAIT** input, and it must be asserted for a minimum of one cycle. If a waited transfer is terminated before it has completed successfully, then an AHB error response is generated, and the WaitToutErr flag in the bank status register is asserted. As an external wait stops any external transfers being performed on the external bus, the SMC generates an error response when a transfer is requested to any external location and the external wait input is still asserted from a previous transfer. This continues until the waited transfer is complete (**SMWAIT** deasserted), and then operation continues as normal.

The WaitStatus bit of the bank status register can be used to check the current status of **SMWAIT** after a terminated transfer.

SMWAIT timing diagrams

Figure 2-21 shows the timing for an externally waited read transfer, taking two cycles for the wait to be asserted, and two cycles for the wait to be deasserted. The synchronization of the asynchronous **SMWAIT** input adds a further two clock cycles onto the timing of the transfer.

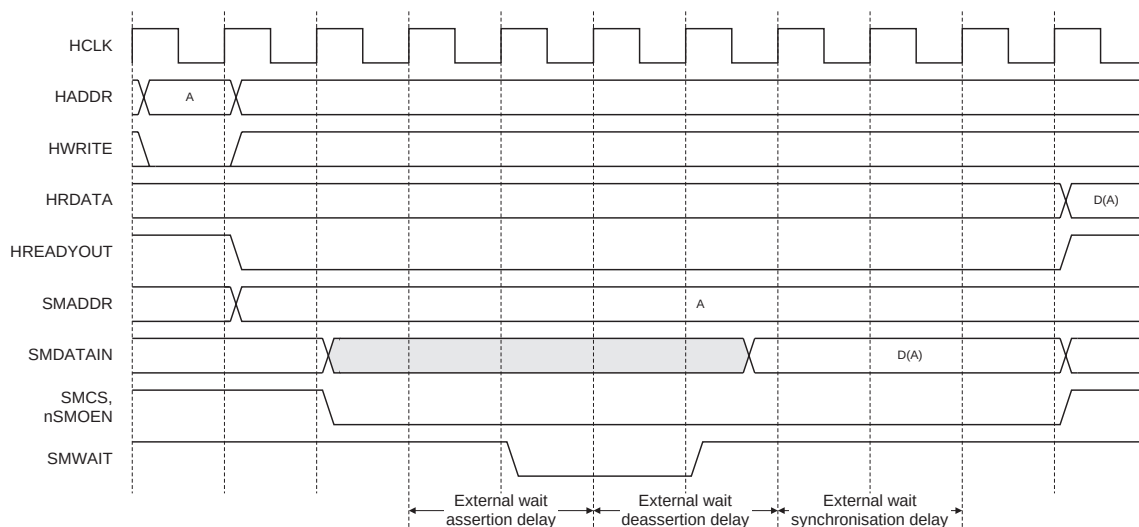


Figure 2-21 External wait timed read transfer

Figure 2-22 shows the timing for an externally waited write transfer, taking two cycles for the wait to be asserted, and two cycles for the wait to be deasserted. An additional cycle is needed at the end of the transfer over a read transfer to allow the deassertion of the write enable before the chip select. An externally waited transfer is also waited on the AHB (unlike a standard write transfer), which allows an error response due to a timeout to be generated correctly.

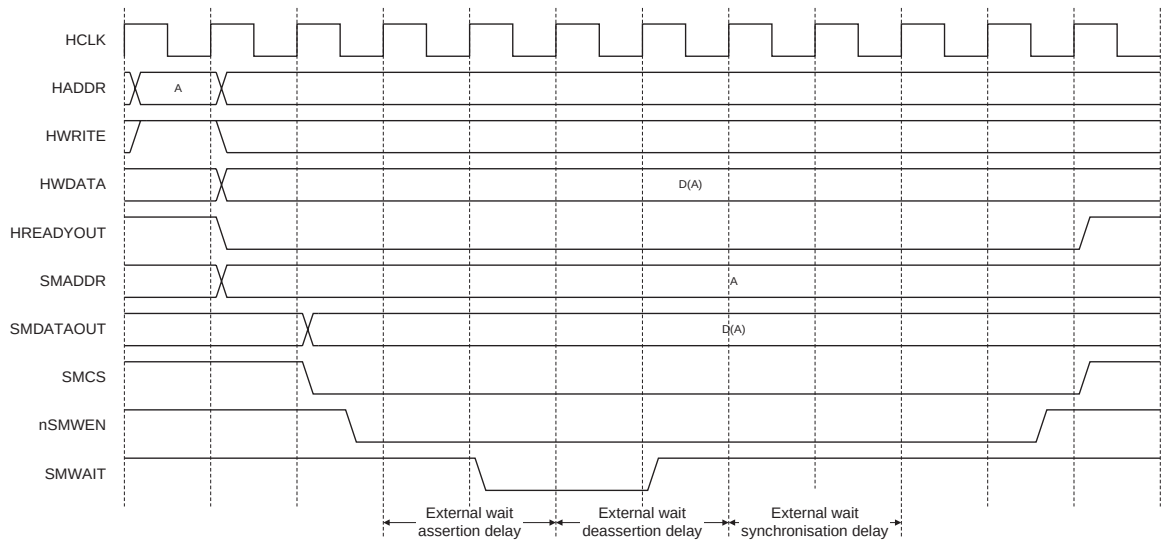


Figure 2-22 External wait timed write transfer

Figure 2-23 shows the timing for an aborted externally waited transfer.

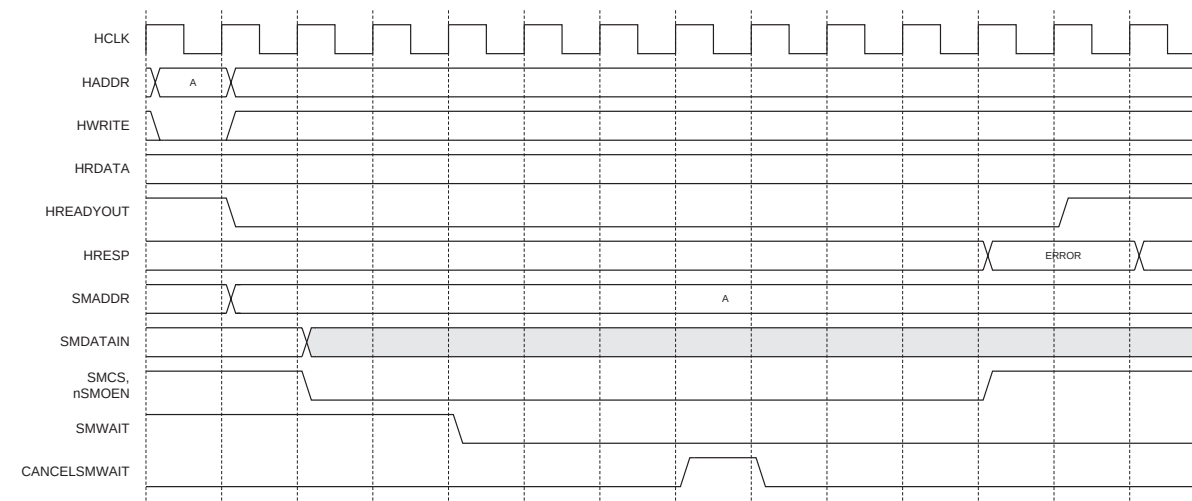


Figure 2-23 External Wait timed read transfer with external abort

2.2.9 Byte lane control

The PrimeCell SMC generates byte lane control signals **nSMBLS[3:0]** according to:

- little or big-endian operation
- AMBA transfer width (indicated by **HSIZE[2:0]**)
- external memory bank data bus width, defined within each configuration register
- external memory bank type, being either byte, halfword or word
- the decoded **HADDR[1:0]** value for write accesses only.

Word transfers are the largest size transfers supported by the PrimeCell SMC, any access attempted with a size greater than a word causes the error response to be generated.

Each memory bank can either be 8, 16, or 32 bits wide. The memory configuration for a particular memory bank determines how the **nSMWEN** and **nSMBLS** signals are connected to provide byte, halfword, and word access. For read accesses, it is necessary to control the **nSMBLS** signals by driving them either all HIGH, or all LOW.

This control is achieved by programming the *Read Byte Lane Enable* (RBLE) bit within each control register. The following two sections explain why different connections in respect of **nSMWEN** and **nSMBLS[3:0]** are needed for different memory configurations.

Accesses to memory banks constructed from 8-bit or non byte-partitioned memory devices

For memory banks constructed from 8-bit or non byte-partitioned memory devices, it is important that the RBLE bit is cleared to zero within the respective memory bank configuration register. This forces all **nSMBLS[3:0]** lines HIGH during a read access to that particular bank, as the byte lane selects are connected to the device write enables.

Figure 2-24 on page 2-34 shows 8-bit memory devices being used to configure memory banks that are 8, 16 and 32 bits wide. In each of these configurations, the **nSMBLS[3:0]** signals are connected to the write enable (**nWE**) inputs of each 8-bit memory.

————— Note —————

The **nSMWEN** signal from the PrimeCell SMC is not used in this configuration.

For write transfers, the relevant **nSMBLS[3:0]** byte lane signals are asserted LOW, and steer the data to the addressed bytes.

For read transfers, all **nSMBLS[3:0]** lines are deasserted HIGH, which allows the external bus to be defined for at least the width of the accessed memory.

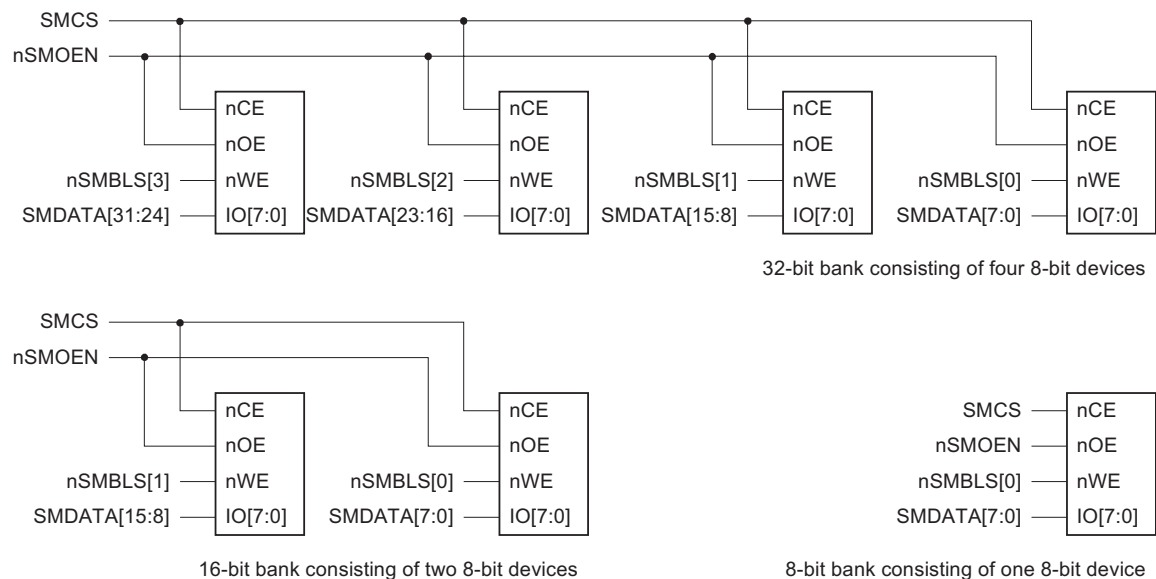


Figure 2-24 Memory banks constructed from 8-bit memory

Accesses to memory banks constructed from 16 or 32-bit memory devices

For memory banks constructed from 16 or 32-bit memory devices, it is important that the RBLE bit is set to one within the respective memory bank control register. This asserts all **nSMBLS[3:0]** lines LOW during a read access to that particular bank, as during a read all bytes of the devices must be selected to avoid undriven byte lanes on the read data value.

For 16 and 32-bit wide memory devices, byte select signals exist and these must be appropriately controlled, as shown in Figure 2-25 and Figure 2-26.

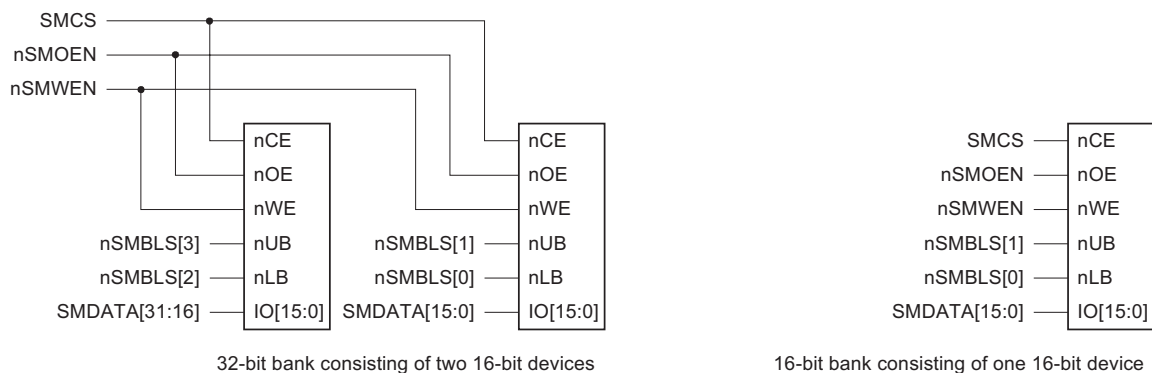


Figure 2-25 Memory banks constructed from 16-bit memory

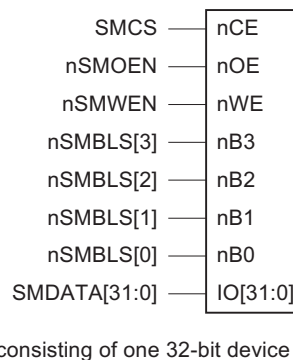


Figure 2-26 Memory banks constructed from 32-bit memory

Figure 4-24 shows connections for a typical memory system with different data width memory devices.

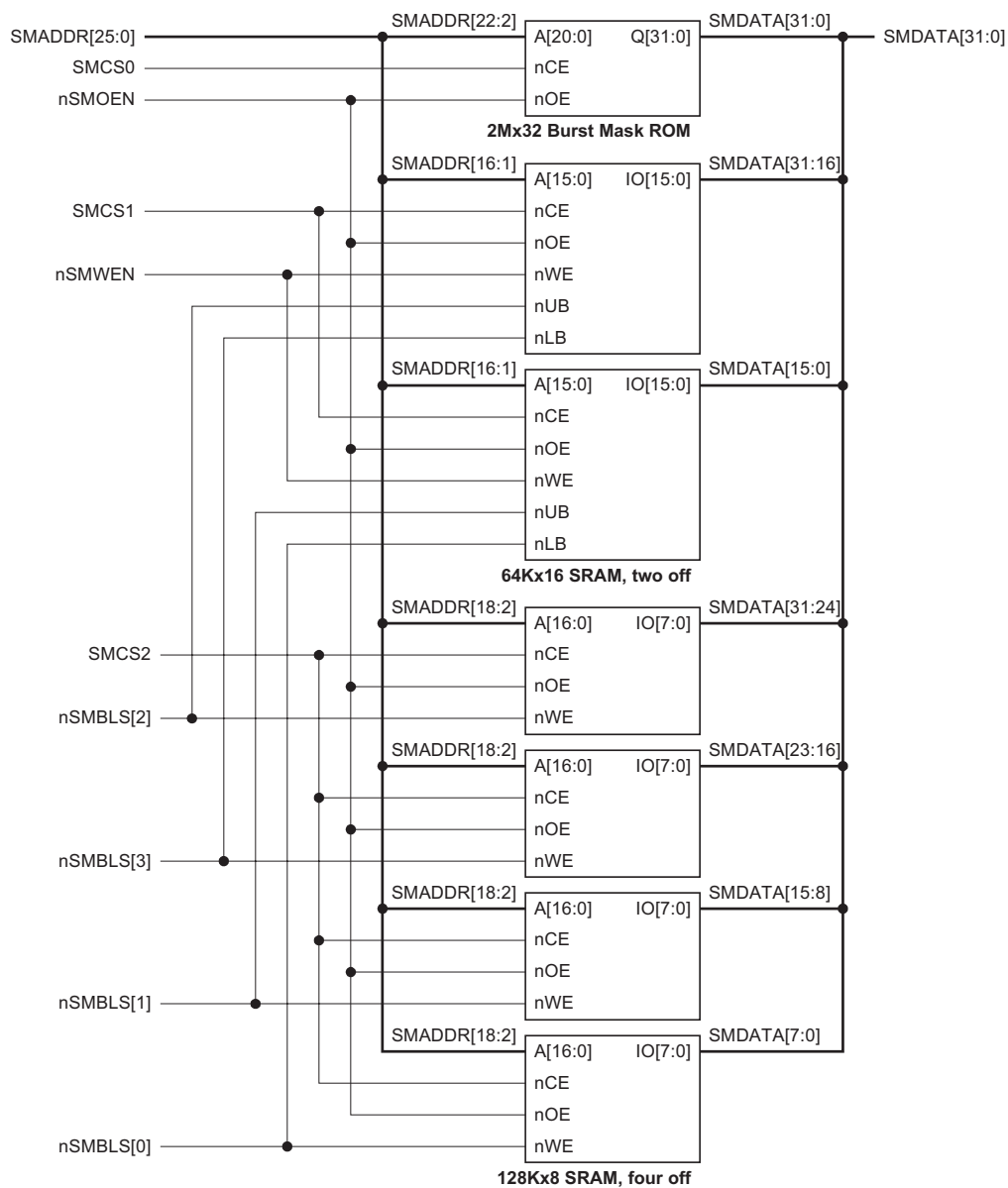


Figure 2-27 Typical memory connection diagram

Elimination of floating bytes on the external interface

The PrimeCell SMC uses the programmed external memory width of each bank and the endianness to determine which remaining bytes it needs to drive to ensure that the external bus is never floating.

The input/output pads cells for the external write data bus lines are controlled by **nSMDATAEN[3:0]** according to Table 2-4. Data is driven out on **SMDATAOUT** when **nSMDATAEN** is asserted LOW.

Table 2-4 SMDATAOUT controlled by nSMDATAEN

nSMDATAEN	SMDATAOUT
0	7:0
1	15:8
2	23:16
3	31:24

Byte lane control and data bus steering for little and big-endian configurations

Table 2-5 on page 2-38 to Table 2-16 on page 2-45 show the relationship of signals **HSIZE[2:0]**, **HADDR[1:0]**, **SMADDR[1:0]**, and **nSMBLS[3:0]**, and mapping of data between the AHB system data bus and external memory data bus.

Table 2-5 Little-endian read, 8-bit external bus

Access: Read, little-endian, 8-bit external bus				External data mapping on to system data bus			
Internal transfer width	HSIZE[1:0]	HADDR[1:0]	SMADDR[1:0]	31:24	23:16	15:8	7:0
Word (4 transfers)	10	xx	11	7:0	-	-	-
	10	xx	10	-	7:0	-	-
	10	xx	01	-	-	7:0	-
	10	xx	00	-	-	-	7:0
Halfword (2 transfers)	01	1x	11	7:0	-	-	-
	01	1x	10	-	7:0	-	-
Halfword (2 transfers)	01	0x	01	-	-	7:0	-
	01	0x	00	-	-	-	7:0
Byte	00	11	11	7:0	-	-	-
Byte	00	10	10	-	7:0	-	-
Byte	00	01	01	-	-	7:0	-
Byte	00	00	00	-	-	-	7:0

Table 2-6 Little-endian read, 16-bit external bus

Access: Read, little-endian, 16-bit external bus				External data mapping on to system data bus			
Internal transfer width	H SIZE[1:0]	H ADDR[1:0]	S M ADDR[1:0]	31:24	23:16	15:8	7:0
Word (2 transfers)	10	xx	1x	15:8	7:0	-	-
	10	xx	0x	-	-	15:8	7:0
Halfword	01	1x	1x	15:8	7:0	-	-
Halfword	01	0x	0x	-	-	15:8	7:0
Byte	00	11	1x	15:8	-	-	-
Byte	00	10	1x	-	7:0	-	-
Byte	00	01	0x	-	-	15:8	-
Byte	00	00	0x	-	-	-	7:0

Table 2-7 Little-endian read, 32-bit external bus

Access: Read, little-endian, 32-bit external bus				External data bus mapping on to system data bus			
Internal transfer width	H SIZE[1:0]	H ADDR[1:0]	S M ADDR[1:0]	31:24	23:16	15:8	7:0
Word	10	xx	xx	31:24	23:16	15:8	7:0
Halfword	01	1x	xx	31:24	23:16	15:8	7:0
Halfword (2 transfers)	01	0x	xx	31:24	23:16	15:8	7:0
Byte	00	11	xx	31:24	23:16	15:8	7:0
Byte	00	10	xx	31:24	23:16	15:8	7:0
Byte	00	01	xx	31:24	23:16	15:8	7:0
Byte	00	00	xx	31:24	23:16	15:8	7:0

Table 2-8 Little-endian write, 8-bit external bus

Access: Write, little-endian, 8-bit external bus					System data mapping on to external data bus			
Internal transfer width	HSIZE[1:0]	HADDR[1:0]	SMADDR [1:0]	nSMBLS [3:0]	31:24	23:16	15:8	7:0
Word (4 transfers)	10	xx	11	1110	-	-	-	31:24
	10	xx	10	1110	-	-	-	23:16
	10	xx	01	1110	-	-	-	15:8
	10	xx	00	1110	-	-	-	7:0
Halfword (2 transfers)	01	1x	11	1110	-	-	-	31:24
	01	1x	10	1110	-	-	-	23:16
Halfword (2 transfers)	01	0x	01	1110	-	-	-	15:8
	01	0x	00	1110	-	-	-	7:0
Byte	00	11	11	1110	-	-	-	31:24
Byte	00	10	10	1110	-	-	-	23:16
Byte	00	01	01	1110	-	-	-	15:8
Byte	00	00	00	1110	-	-	-	7:0

Table 2-9 Little-endian write, 16-bit external bus

Access: Write, little-endian, 16-bit external bus					System data mapping on to external data bus			
Internal transfer width	HSIZE[1:0]	HADDR[1:0]	SMADDR [1:0]	nSMBLS [3:0]	31:24	23:16	15:8	7:0
Word (2 transfers)	10	xx	1x	1100	-	-	31:24	23:16
	10	xx	0x	1100	-	-	15:8	7:0
Halfword	01	1x	1x	1100	-	-	31:24	23:16
Halfword	01	0x	0x	1100	-	-	15:8	7:0
Byte	00	11	1x	1101	-	-	31:24	-
Byte	00	10	1x	1110	-	-	-	23:16
Byte	00	01	0x	1101	-	-	15:8	-
Byte	00	00	0x	1110	-	-	-	7:0

Table 2-10 Little-endian write, 32-bit external bus

Access: write, little-endian, 32-bit external bus					System data mapping on to external data bus			
Internal transfer width	HSIZE[1:0]	HADDR[1:0]	SMADDR [1:0]	nSMBLS [3:0]	31:24	23:16	15:8	7:0
Word	10	xx	xx	0000	31:24	23:16	15:8	7:0
Halfword	01	1x	xx	0011	31:24	23:16	-	-
Halfword	01	0x	xx	1100	-	-	15:8	7:0
Byte	00	11	xx	0111	31:24	-	-	-
Byte	00	10	xx	1011	-	23:16	-	-
Byte	00	01	xx	1101	-	-	15:8	-
Byte	00	00	xx	1110	-	-	-	7:0

Table 2-11 Big-endian read, 8-bit external bus

Access: Read, big-endian, 8-bit external bus				External data mapping on to system data bus			
Internal transfer width	HSIZE[1:0]	HADDR[1:0]	SMADDR[1:0]	31:24	23:16	15:8	7:0
Word (4 transfers)	10	xx	11	-	-	-	7:0
	10	xx	10	-	-	7:0	-
	10	xx	01	-	7:0	-	-
	10	xx	00	7:0	-	-	-
Halfword (2 transfers)	01	1x	11	-	-	-	7:0
	01	1x	10	-	-	7:0	-
Halfword (2 transfers)	01	0x	01	-	7:0	-	-
	01	0x	00	7:0	-	-	-
Byte	00	11	11	-	-	-	7:0
Byte	00	10	10	-	-	7:0	-
Byte	00	01	01	-	7:0	-	-
Byte	00	00	00	7:0	-	-	-

Table 2-12 Big-endian read, 16-bit external bus

Access: Read, big-endian, 16-bit external bus				External data mapping on to system data bus			
Internal transfer width	H SIZE[1:0]	H ADDR[1:0]	S M ADDR[1:0]	31:24	23:16	15:8	7:0
Word (2 transfers)	10	xx	1x	-	-	15:8	7:0
	10	xx	0x	15:8	7:0	-	-
Halfword	01	1x	1x	-	-	15:8	7:0
Halfword	01	0x	0x	15:8	7:0	-	-
Byte	00	11	1x	-	-	-	7:0
Byte	00	10	1x	-	-	15:8	-
Byte	00	01	0x	-	7:0	-	-
Byte	00	00	0x	15:8	-	-	-

Table 2-13 Big-endian read, 32-bit external bus

Access: Read, Big-endian, 32-bit external bus				External data mapping on to system data bus			
Internal transfer width	H SIZE[1:0]	H ADDR[1:0]	S M ADDR[1:0]	31:24	23:16	15:8	7:0
Word	10	xx	xx	31:24	23:16	15:8	7:0
Halfword	01	1x	xx	31:24	23:16	15:8	7:0
Halfword (2 transfers)	01	0x	xx	31:24	23:16	15:8	7:0
Byte	00	11	xx	31:24	23:16	15:8	7:0
Byte	00	10	xx	31:24	23:16	15:8	7:0
Byte	00	01	xx	31:24	23:16	15:8	7:0
Byte	00	00	xx	31:24	23:16	15:8	7:0

Table 2-14 Big-endian write, 8-bit external bus

Access: Write, big-endian, 8-bit external bus					System data mapping on to external data bus			
Internal transfer width	H SIZE[1:0]	H ADDR[1:0]	S M ADDR[1:0]	n S M BLS[3:0]	31:24	23:16	15:8	7:0
Word (4 transfers)	10	xx	11	1110	-	-	-	7:0
	10	xx	10	1110	-	-	-	15:8
	10	xx	01	1110	-	-	-	23:16
	10	xx	00	1110	-	-	-	31:24
Halfword (2 transfers)	01	1x	11	1110	-	-	-	7:0
	01	1x	10	1110	-	-	-	15:8
Halfword (2 transfers)	01	0x	01	1110	-	-	-	23:16
	01	0x	00	1110	-	-	-	31:24
Byte	00	11	11	1110	-	-	-	7:0
Byte	00	10	10	1110	-	-	-	15:8
Byte	00	01	01	1110	-	-	-	23:16
Byte	00	00	00	1110	-	-	-	31:24

Table 2-15 Big-endian write, 16-bit external bus

Access: Write, big-endian, 16-bit external bus					System data mapping on to external data bus			
Internal transfer width	HSIZE[1:0]	HADDR[1:0]	SMADDR [1:0]	nSMBLS [3:0]	31:24	23:16	15:8	7:0
Word (2 transfers)	10	xx	1x	1100	-	-	15:8	7:0
	10	xx	0x	1100	-	-	31:24	23:16
Halfword	01	1x	1x	1100	-	-	15:8	7:0
Halfword	01	0x	0x	1100	-	-	31:24	23:16
Byte	00	11	1x	1110	-	-	-	7:0
Byte	00	10	1x	1101	-	-	15:8	-
Byte	00	01	0x	1110	-	-	-	23:16
Byte	00	00	0x	1101	-	-	31:24	-

Table 2-16 Big-endian write, 32-bit external bus

Access: Write, big-endian, 32-bit external bus					System data mapping on to external data bus			
Internal transfer width	HSIZE[1:0]	HADDR[1:0]	SMADDR [1:0]	nSMBLS [3:0]	31:24	23:16	15:8	7:0
Word	10	xx	xx	0000	31:24	23:16	15:8	7:0
Halfword	01	1x	xx	1100	-	-	15:8	7:0
Halfword	01	0x	xx	0011	31:24	23:16	-	-
Byte	00	11	xx	1110	-	-	-	7:0
Byte	00	10	xx	1101	-	-	15:8	-
Byte	00	01	xx	1011	-	23:16	-	-
Byte	00	00	xx	0111	31:24	-	-	-

2.3 Memory shadowing

This section describes how the SMC can be configured to allow boot ROM to be available instead of RAM at the base memory location immediately after reset, and how the boot ROM device size can be configured at reset.

2.3.1 Booting from ROM after reset

There are two possible configurations for a system that uses boot ROM:

- internal ROM
- external ROM.

For a system with internal ROM, the AHB Decoder handles selecting this device after reset to allow the processor to start executing from address 0x00000000. For a system with external ROM, the external memory controller must ensure that this device is available at address 0x00000000 no matter which chip select it is connected to.

In a system with internal RAM and external ROM, the most likely memory map configuration is to have the internal RAM at address 0x00000000. Because this RAM is likely to be quite small, it is useful to overlap the normally larger external RAM with this internal RAM, so the external RAM is located on bank zero and also at address 0x00000000. This allows an easy flow from internal RAM to external RAM, and makes it possible for the software to continue accessing the RAM without having to write to a completely different location if it exceeds the size of the internal RAM.

To accommodate this overlap of internal and external RAMs, the external boot ROM must be on a bank other than bank zero. For the Primecell SMC this boot ROM is available on bank seven.

The SMC therefore provides a **REMAP** input that is used to control the shadowing of the **SMCS[7]** memory area onto the **SMCS[0]** memory area after reset (**REMAP** = 0). This makes the memory at **SMCS[0]** unavailable until the system sets the **REMAP** input. Once the **REMAP** input has been cleared, the **SMCS[7]** memory area is only available at **SMCS[7]**, and the **SMCS[0]** memory area is available for normal use.

Chip selects **SMCS[1]** to **SMCS[6]** are unaffected by the **REMAP** input and are always available for access.

————— Note —————

The **REMAP** input must be generated as part of the users system, probably as part of the system controller logic, and when LOW indicates that the system requires external boot ROM to be available at address 0x00000000. If the remapping function is not required, then **REMAP** must be tied HIGH to disable memory bank shadowing.

2.3.2 External bank SMCS7 size configuration

The SMC allows the size of bank seven (selected with **SMCS[7]**) to be configured at reset using the external control pins **SMMWCS7[1:0]**. This feature is used to allow the boot memory size to be chosen at reset.

Bank seven can therefore always have the correct setting, and no additional software programming of the bank seven control register is required. However, the MW bits of **SMBCR7** are still programmable, the same as the rest of the bank configuration registers. If you choose not to reconfigure the size of bank seven (by tying the pins to 11), then the bank size defaults to 8-bits.

Note

All other memory bank sizes have default settings that are set in the hardware, but can be programmed to the required values using software.

The configuration of the bits used is shown in Table 2-17.

Table 2-17 External size configuration values for bank seven

SMMWCS7[1:0]	Memory width
00	8-bit
01	16-bit
10	32-bit
11	No change

2.4 Test interface controller operation

The *Test Interface Controller* (TIC) is a state machine that provides an AMBA AHB bus master for system test. It reads test write and address data from the external data bus **SMDATA**, and drives the external bus with test read data, allowing the use of only one set of output tristate buffers onto **SMDATA**.

The TIC is used to convert externally applied test vectors into internal transfers on the AHB bus. A three-wire external handshake protocol is used, with two inputs controlling the type of vector that is applied and a single output that indicates when the next vector can be applied.

Typically the TIC is the highest priority AMBA bus master, which ensures test access under all conditions.

The TIC model supports address incrementing and control vectors. This means that the address for burst transfers can automatically be generated by the TIC.

The TIC is described in the following subsections:

- *Function and operation of module* on page 2-48
- *Test vector types* on page 2-49
- *Control vectors* on page 2-50
- *Test vector sequences* on page 2-51
- *Data bus interface operation* on page 2-58.

2.4.1 Function and operation of module

The TIC operates as a standard AHB bus master during system test when the external test pins show that the system is required to enter test mode. In this mode, the requests control of the AHB, and when granted uses the AHB to perform system tests.

Table 2-18 shows the operation of the external test pins to change the TIC mode from normal operation into test mode.

Table 2-18 Test control signals during normal operation

TESTREQA	TESTREQB	TESTACK	Description
0	-	0	Normal operation
1	-	0	Enter test mode request
-	-	1	Test mode entered

During system test the external test pins are used to control the operation of the TIC. The operation of these pins is shown in Table 2-19.

Table 2-19 Test control signals during test operation

TESTREQA	TESTREQB	TESTACK	Description
-	-	0	Current access incomplete
1	1	1	Address vector or Control vector or Turnaround vector
1	0	1	Write vector
0	1	1	Read vector
0	0	1	Exit test mode

On entry into test mode the TIC indicates that it has switched to the test clock input by asserting the **TESTACK** signal.

2.4.2 Test vector types

There are five types of test vector associated with the test interface:

- Address vector** The address for all subsequent read and write transfers is sampled by the TIC.
- Write vector** The TIC performs an AHB write cycle, using the write data currently driven onto the external data bus.
- Read vector** The TIC performs an AHB read cycle, driving the read data onto the external data bus when it becomes valid.
- Control vector** Internal TIC registers are set, that control the types of read and write transfers that are performed.
- Turnaround vector** Used between a read cycle and a write cycle to avoid clashes on the external data bus.

The address, control and turnaround vectors are all indicated by the same value on the **TESTREQA** and **TESTREQB** signals. You can use the following rules to determine which type of vector is being applied:

- a read vector, or burst of read vectors, is followed by two turnaround vectors
- when a single address or control vector is applied it is an address vector
- when multiple address or control vectors are applied they are all address vectors, apart from the last which is a control vector.

2.4.3 Control vectors

The control vector is used to determine the types of transfer the TIC can perform, by setting the values of the **HSIZETIC**, **HPROTTIC**, and **HLOCKTIC** AHB master outputs.

The default TIC bus master transfer type is:

32-bit transfer width

HSIZETIC[2:0] signifies word transfer.

Privileged system access

HPROTTIC[3:0] signifies supervisor data access, uncacheable and unbufferable.

Bit 0 of the control vector is used to indicate if the control vector is valid. Therefore if a control vector is applied with bit 0 LOW, the vector is ignored and does not update the control information. This mechanism allows address vectors that have bit 0 LOW to be applied for many cycles without updating the control information.

Although the default settings are sufficient for testing many systems, control vectors can be used to change the control signals of the transfer, and can also be used to select whether the TIC must generate fixed addresses or incrementing addresses.

Table 2-20 defines the bit positions of the control vector. The control vector bit definitions are designed to be backwards compatible with earlier versions of the TIC and therefore not all of the control bits are in obvious positions.

Table 2-20 Control vector bit definitions

Bit position	Description
0	Control vector valid
1	Reserved
2	HSIZETIC[0]
3	HSIZETIC[1]
4	HLOCKTIC
5	HPROTTIC[0]
6	HPROTTIC[1]
7	Address increment enable
8	Reserved
9	HPROTTIC[2]
10	HPROTTIC[3]

There is no mechanism to control the types of burst that the TIC can perform and only incrementing bursts of an undefined length are supported. The TIC only supports 8-bit, 16-bit, and 32-bit transfers and therefore **HSIZETIC[2]** cannot be altered and is always LOW.

To support burst accesses using the test interface the TIC can support incrementing of the bus address. The TIC increments eight address bits and the address range that can be covered by this incrementer depends on the size of the transfers being performed.

The control vector provides a mechanism to enable and disable the address incrementer within the TIC. This allows burst accesses to incremental addresses, as would be used for testing internal RAM. Alternatively the address increment can be disabled, so that successive accesses of a burst occur to the same address, as would be required to continually read from a single peripheral register.

The address incrementer is disabled by default and must be enabled using a control vector prior to use.

Note

The control vector is primarily used to change signals that have the same timing as the address bus. However the control vector also allows the lock signal to be changed, which is actually required before the locked transfer commences. If the **HLOCKTIC** signal is used during testing it must be set a transfer before it is required. This difference in timing on the **HLOCKTIC** signal can in some cases cause an additional transfer to be locked both before and after the sequence that must in fact be locked.

2.4.4 Test vector sequences

This section describes:

- *Entering test mode* on page 2-52
- *Write vectors* on page 2-53
- *Read vectors* on page 2-54
- *Control vector* on page 2-55
- *Burst vector* on page 2-56
- *Read-to-write and write-to-read transfers* on page 2-56
- *Exiting test mode* on page 2-57.

Entering test mode

In normal operating mode **TESTREQA** is LOW, indicating that test access is not required. Entering test mode allows test vectors to be applied externally that cause transfers on the internal bus. Figure 2-28 shows the test start sequence.

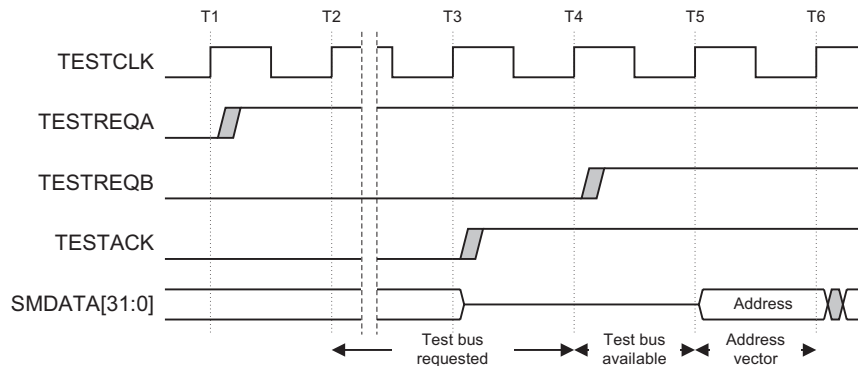


Figure 2-28 Test start sequence

The following sequence, as shown in Figure 2-28, is required in order to enter test mode:

1. **TESTREQA** is asserted to request test bus access.
2. Test mode is entered when the TIC is granted the internal bus and this is indicated by the assertion of the **TESTACK** signal.
3. When test mode has been entered **TESTREQB** is asserted to initiate an address vector.
4. The TIC does not perform any internal transfers until a valid address vector has been applied.

A synchronous tester is not expected to poll **TESTACK** for the bus. Usually the **TESTREQA** signal is asserted for a minimum number of cycles guaranteed to gain access to the bus (completion of the longest wait-state peripheral access or the maximum number of cycles for all bus masters to have completed their current instruction).

Write vectors

Figure 2-29 shows the sequence of events when applying a set of write test vectors. Initially an address vector is applied and this is followed by a write test vector.

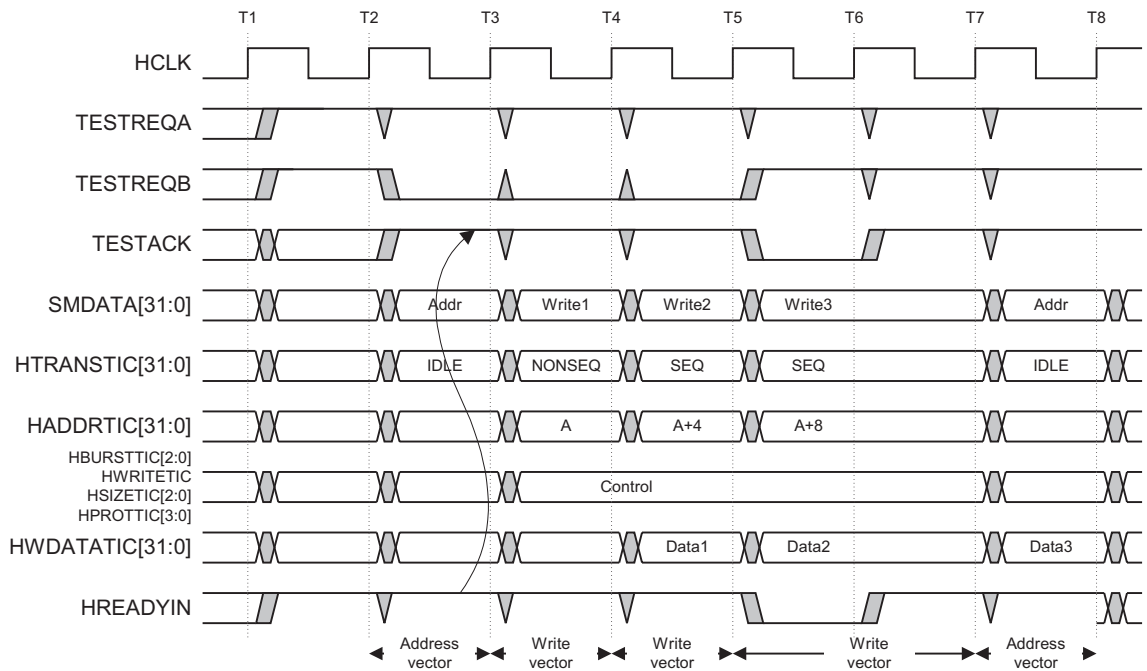


Figure 2-29 Write test vectors

The **TESTREQA** and **TESTREQB** signals are pipelined and are used to indicate what type of vector is applied in the following cycle.

The TIC samples the address and the **TESTREQA** and **TESTREQB** signals at time T3 and following this it can initiate the appropriate transfer on the AHB. In the following cycle the write data is driven onto **SMDATA** and it is then sampled on the following clock edge, T4, and driven onto the internal bus.

If the internal transfer is not able to complete then the **TESTACK** signal is driven LOW and this indicates that the external test vector must be applied for another cycle.

Read vectors

Read transfers are more complex because they require **SMDATA** to be driven in the opposite direction, and therefore additional cycles are required to prevent bus clash when changing between different drivers of **SMDATA**. Figure 2-30 shows a typical test sequence for reads.

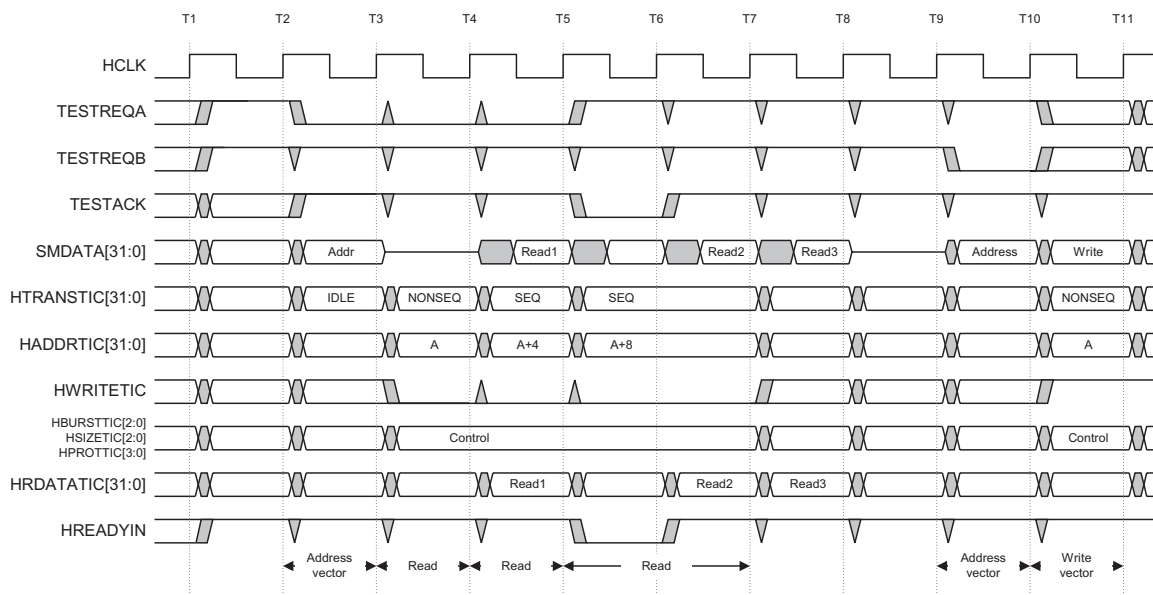


Figure 2-30 Read test vectors

The **TESTREQA** and **TESTREQB** signals are used in the same way as for a write transfer. Initially **TESTREQA** and **TESTREQB** are used to apply an address vector and then in the following cycle they are used to indicate that a read transfer is required. For the first cycle of a read **SMDATA** must be tristated, which ensures that the external tester driving **SMDATA** has an entire cycle to tristate its buffers before the TIC enables the on-chip buffers to drive out the read data.

At the end of a burst of reads it is necessary to allow time for bus turnaround. In this case the TIC must turn off the internal buffers and an entire cycle is allowed before the external test equipment starts to drive **SMDATA**.

The end of a burst of reads is indicated by both **TESTREQA** and **TESTREQB** being HIGH, as for an address vector. They must indicate an address vector for two cycles, which allows for the turnaround cycle at the start of the burst and also the turnaround cycle at the end of the burst.

Control vector

The operation of the TIC can be modified by the use of a control vector. Whenever more than one address vector is applied in succession then the last vector is considered to be a control vector and is not sampled as an address value. Bit 0 of the control vector is used to determine whether or not the control vector must be considered valid, which allows multiple address vectors to be applied without changing the control information. Figure 2-31 shows the process of inserting a control vector.

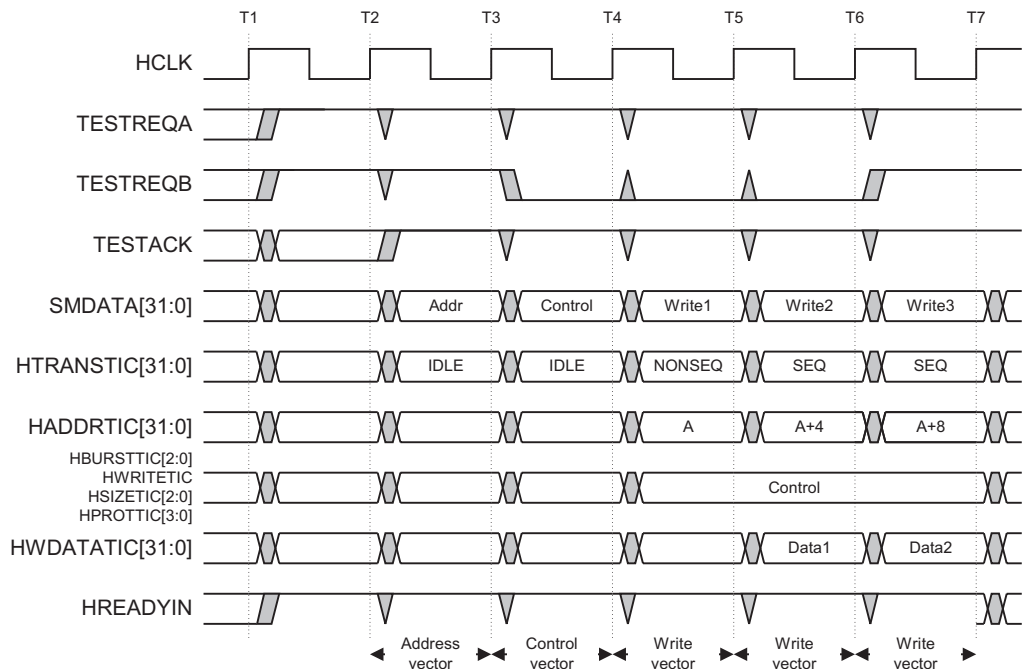


Figure 2-31 Control vector

At time T4 the TIC can determine that **SMDATA** contains a control vector. This is because the previous cycle was an address vector and **TESTREQA** and **TESTREQB** are indicating that the following cycle is either a read or a write and therefore the current cycle must be a control vector.

Burst vector

The examples of read and write transfers in Figure 2-30 on page 2-54 and Figure 2-31 show how additional transfers can be used to form burst transfers on the bus. The TIC has limited capabilities for burst transfers and can only perform undefined length incrementing bursts.

The TIC contains an 8-bit incrementer and if an attempt is made to perform a burst that crosses the incrementer boundary, then the address simply wraps and the TIC signals the transfer as **NONSEQUENTIAL**. The exact boundary at which this occurs depends on the size of the transfer. For word transfers the incrementer overflows at 1KB boundaries, for halfword transfers it overflows at 512-byte boundaries and for byte transfers the overflow occurs at 256-byte boundaries.

Read-to-write and write-to-read transfers

It is possible to switch between read transfers and write transfers without applying a new address vector. Usually this is done with the address incrementer disabled, so that both the read transfers and the write transfers are to the same address. It is also possible to do this with the incrementer enabled if the test circumstances require it.

When moving from a read transfer to a write transfer it is necessary to allow two cycles for bus handover and therefore **TESTREQA** and **TESTREQB** must signal an address vector for two cycles after the read. This does not cause the address to be changed unless it is followed by a third address vector.

Figure 2-32 shows the sequence of events.

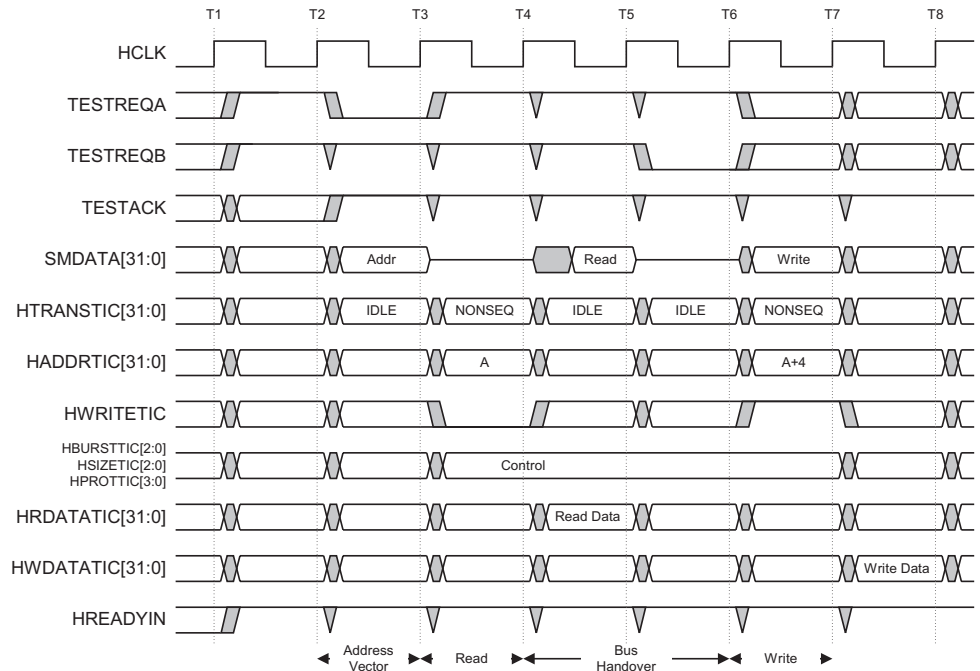


Figure 2-32 Read vector followed by a write vector

Exiting test mode

Test mode is exited using the following sequence:

1. Apply a single cycle address vector, which causes an IDLE cycle internally. This ensures any internal transfers are complete and an Address-Only transfer is performed on the internal bus.
2. **TESTREQA** and **TESTREQB** are both driven LOW to indicate that test mode is to be exited.
3. When the test interface has been configured for normal system operation, **TESTACK** goes LOW to indicate that test mode has been exited.

It is important that test mode can be entered and exited cleanly so that the TIC can be used for diagnostic test during system operation, as well as during production testing.

2.5 Data bus interface operation

The *Data Bus Interface* (DBI) allows both parts of the SMC and the additional memory controller to share the external address and data buses, by only selecting one device to drive them at a time. The **SMADDR/MCADDR**, **SMDATAOUT/TBUSOUTEBI/MCDATAOUT** and **nSMDATAEN/TICREADEBI/MCDATAEN** lines are multiplexed by this block. All other signals pass directly to the memory controllers and the test controller, including the external data input.

A simple priority based request and grant system is used, with the TIC the highest priority device (as it must always be possible to enter test mode), and the PrimeCell SMC the lowest priority device. The bus interface degrades a lower priority device when a higher priority device requests control of the bus, but the higher priority device only gains control of the bus once the lower priority device finishes the current transfer and releases control of the bus. The SMC is granted control of the bus by default, reducing the access time during normal operation.

An example of the timing of the bus interface is shown in Figure 2-33.

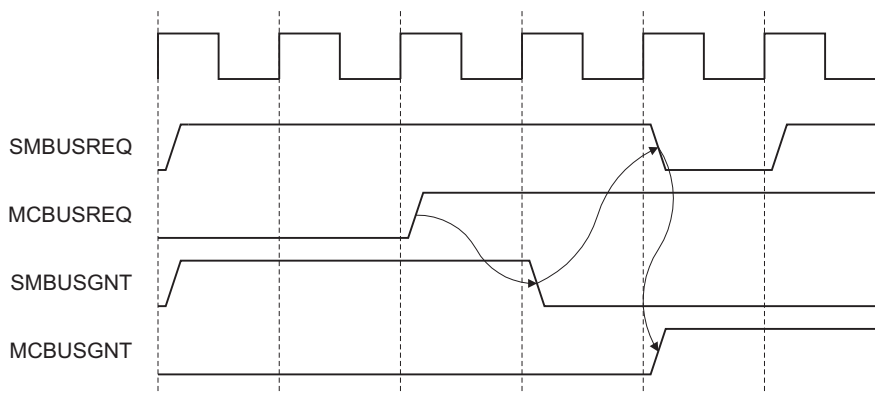


Figure 2-33 Example of bus interface timing

When the SMC requests the bus the other higher priority device is not requesting control of the bus (**MCBUSREQ** deasserted), so the SMC is granted immediately. However, during the course of the SMC transfer, the higher priority device requests the bus (**MCBUSREQ** asserted) and hence the SMC grant line is taken LOW by the DBI in the following cycle. This indicates to the SMC that it must complete the current external transfer and deassert **SMBUSREQ**. Once the SMC has reached the end of this transfer it sets the **SMBUSREQ** output LOW, that indicates to the DBI that it can grant control of the external bus to the other higher priority device. If the SMC still has further transfers to complete (for example it might have terminated a word access from a byte-wide memory after two bytes), then it can reassert the **SMBUSREQ** signal one clock cycle after deasserting it and wait to be regranted before continuing the transfer.

The timing in Figure 2-33 on page 2-58 shows the minimum time it takes for the additional memory controller to gain control of the bus if the SMC is currently granted and performing a transfer. The additional controller must wait a minimum of two cycles between asserting its bus request and the DBI asserting the bus grant. It takes one cycle for the SMC to be degranted, and a minimum of one cycle for the SMC to deassert its bus request output and for the additional controller to gain control of the bus. The maximum time that can occur before gaining control of the external bus depends on the system, as an externally waited SMC transfer has no time limit, and depends on the device driving **CANCELSMWAIT** to set a maximum time limit for a waited transfer.

2.6 Using the SMC with an external bus multiplexer or SDRAM controller

The SMC is designed to allow the use of an external bus multiplexer (instead of the internal DBI) to control the switching of the external data and address buses, for example if the SMC is to be used with an SDRAM controller which needs a bus multiplexer that can handle the necessary synchronization retiming.

The **EXTBUSMUX** input must be tied high so that the internal DBI is bypassed and the external bus multiplexer is used instead. This enables the external request and grant ports instead of the internal DBI connections.

If the external bus multiplexer contains retiming registers on the address output path (for example when used with a synchronous memory controller such as an SDRAM controller), ensure that the **WSTWEN** values for all writeable register banks are set to a minimum of one, and that the **WST2** values are also set to a minimum of one for the same banks. This is because the write enable must only be valid after the address output is stable to make sure that the write is to the correct address. This is only required for writes due to the half a clock cycle timing between the address being stable and the write enable being asserted. As reads use full cycle timing then no special settings are required.

The SMC and TIC bus request and grant ports should be connected to the external bus multiplexer, ensuring that if the TIC is used it is connected to the ports for the highest priority device as it must always be able to enter test mode. The SMC address, data and data enable ports and the TIC data and data enable ports should also be connected.

———— **Note** ————

The additional memory controller ports of the SMC are not used when the internal DBI is bypassed. Any additional controller should be connected directly to the external bus multiplexer.

When the SMC is used with an SDRAM controller, care must be taken to ensure that the SMC does not take control of the bus for longer than the refresh timing of the SDRAM. Because externally waited transfers have undefined lengths, the system must be able to use the **CANCEL_{SM}WAIT** signal to cancel any externally waited transfer and allow the SDRAM refresh to occur.

Chapter 3

Programmer's Model

This chapter describes the registers of the ARM PrimeCell Static Memory Controller (PL092) and provides details required when programming the microcontroller. It contains the following:

- *About the programmer's model* on page 3-2
- *PrimeCell SMC registers* on page 3-3.

3.1 About the programmer's model

The base addresses for the PrimeCell SMC bank configuration registers and memory banks can be configured to suit each particular system implementation. The base addresses must be configured by constant definitions in the *Hardware Description Language* (HDL) code for the AMBA address decoder. The base addresses are not software-programmable.

The PrimeCell SMC registers and memory banks have fixed address offsets from the base addresses described above.

3.2 PrimeCell SMC registers

The PrimeCell SMC registers are shown in Table 3-1.

Table 3-1 PrimeCell SMC register summary

Address	Type	Width	Reset value	Name	Description
SMC RegBase + 0x00	Read/write	4	0xF	SMBIDCYR0	Idle cycle control register for memory bank 0. See Table 3-2 on page 3-9.
SMC RegBase + 0x04	Read/write	5	0x1F	SMBWST1R0	Wait state 1 control register for memory bank 0. See Table 3-3 on page 3-10.
SMC RegBase + 0x08	Read/write	5	0x1F	SMBWST2R0	Wait state 2 control register for memory bank 0. See Table 3-4 on page 3-11.
SMC RegBase + 0x0C	Read/write	4	0x0	SMBWSTOENR0	Output enable assertion delay control register for memory bank 0. See Table 3-5 on page 3-11.
SMC RegBase + 0x10	Read/write	4	0x1	SMBWSTWENR0	Write enable assertion delay control register for memory bank 0. See Table 3-6 on page 3-12.
SMC RegBase + 0x14	Read/write	8	0x80	SMBCR0	Control register for memory bank 0. See Table 3-8 on page 3-13.
SMC RegBase + 0x18	Read/write	3	0x0	SMBSR0	Status register for memory bank 0. See Table 3-9 on page 3-15.
SMC RegBase + 0x1C	Read/write	4	0xF	SMBIDCYR1	Idle cycle control register for memory bank 1. See Table 3-2 on page 3-9.
SMC RegBase + 0x20	Read/write	5	0x1F	SMBWST1R1	Wait state 1 control register for memory bank 1. See Table 3-3 on page 3-10.

Table 3-1 PrimeCell SMC register summary (continued)

Address	Type	Width	Reset value	Name	Description
SMC RegBase + 0x24	Read/write	5	0x1F	SMBWST2R1	Wait state 2 control register for memory bank 1. See Table 3-4 on page 3-11.
SMC RegBase + 0x28	Read/write	4	0x0	SMBWSTOENR1	Output enable assertion delay control register for memory bank 1. See Table 3-5 on page 3-11.
SMC RegBase + 0x2C	Read/write	4	0x1	SMBWSTWENR1	Write enable assertion delay control register for memory bank 1. See Table 3-6 on page 3-12.
SMC RegBase + 0x30	Read/write	8	0x00	SMBCR1	Control register for memory bank 1. See Table 3-8 on page 3-13.
SMC RegBase + 0x34	Read/write	3	0x0	SMBSR1	Status register for memory bank 1. See Table 3-9 on page 3-15.
SMC RegBase + 0x38	Read/write	4	0xF	SMBIDCYR2	Idle cycle control register for memory bank 2. See Table 3-2 on page 3-9.
SMC RegBase + 0x3C	Read/write	5	0x1F	SMBWST1R2	Wait state 1 control register for memory bank 2. See Table 3-3 on page 3-10.
SMC RegBase + 0x40	Read/write	5	0x1F	SMBWST2R2	Wait state 2 control register for memory bank 2. See Table 3-4 on page 3-11.
SMC RegBase + 0x44	Read/write	4	0x0	SMBWSTOENR2	Output enable assertion delay control register for memory bank 2. See Table 3-5 on page 3-11.
SMC RegBase + 0x48	Read/write	4	0x1	SMBWSTWENR2	Write enable assertion delay control register for memory bank 2. See Table 3-6 on page 3-12.

Table 3-1 PrimeCell SMC register summary (continued)

Address	Type	Width	Reset value	Name	Description
SMC RegBase + 0x4C	Read/write	8	0x40	SMBCR2	Control register for memory bank 2. See Table 3-8 on page 3-13.
SMC RegBase + 0x50	Read/write	3	0x0	SMBSR2	Status register for memory bank 2. See Table 3-9 on page 3-15.
SMC RegBase + 0x54	Read/write	4	0xF	SMBIDCYR3	Idle cycle control register for memory bank 3. See Table 3-2 on page 3-9.
SMC RegBase + 0x58	Read/write	5	0x1F	SMBWST1R3	Wait state 1 control register for memory bank 3. See Table 3-3 on page 3-10.
SMC RegBase + 0x5C	Read/write	5	0x1F	SMBWST2R3	Wait state 2 control register for memory bank 3. See Table 3-4 on page 3-11.
SMC RegBase + 0x60	Read/write	4	0x0	SMBWSTOENR3	Output enable assertion delay control register for memory bank 3. See Table 3-5 on page 3-11.
SMC RegBase + 0x64	Read/write	4	0x1	SMBWSTWENR3	Write enable assertion delay control register for memory bank 3. See Table 3-6 on page 3-12.
SMC RegBase + 0x68	Read/write	8	0x00	SMBCR3	Control register for memory bank 3. See Table 3-8 on page 3-13.
SMC RegBase + 0x6C	Read/write	3	0x0	SMBSR3	Status register for memory bank 3. See Table 3-9 on page 3-15.
SMC RegBase + 0x70	Read/write	4	0xF	SMBIDCYR4	Idle cycle control register for memory bank 4. See Table 3-2 on page 3-9.
SMC RegBase + 0x74	Read/write	5	0x1F	SMBWST1R4	Wait state 1 control register for memory bank 4. See Table 3-3 on page 3-10.

Table 3-1 PrimeCell SMC register summary (continued)

Address	Type	Width	Reset value	Name	Description
SMC RegBase + 0x78	Read/write	5	0x1F	SMBWST2R4	Wait state 2 control register for memory bank 4. See Table 3-4 on page 3-11.
SMC RegBase + 0x7C	Read/write	4	0x0	SMBWSTOENR4	Output enable assertion delay control register for memory bank 4. See Table 3-5 on page 3-11.
SMC RegBase + 0x80	Read/write	4	0x1	SMBWSTWENR4	Write enable assertion delay control register for memory bank 4. See Table 3-6 on page 3-12.
SMC RegBase + 0x84	Read/write	8	0x80	SMBCR4	Control register for memory bank 4. See Table 3-8 on page 3-13.
SMC RegBase + 0x88	Read/write	3	0x0	SMBSR4	Status register for memory bank 4. See Table 3-9 on page 3-15.
SMC RegBase + 0x8C	Read/write	4	0xF	SMBIDCYR5	Idle cycle control register for memory bank 5. See Table 3-2 on page 3-9.
SMC RegBase + 0x90	Read/write	5	0x1F	SMBWST1R5	Wait state 1 control register for memory bank 5. See Table 3-3 on page 3-10.
SMC RegBase + 0x94	Read/write	5	0x1F	SMBWST2R5	Wait state 2 control register for memory bank 5. See Table 3-4 on page 3-11.
SMC RegBase + 0x98	Read/write	4	0x0	SMBWSTOENR5	Output enable assertion delay control register for memory bank 5. See Table 3-5 on page 3-11.
SMC RegBase + 0x9C	Read/write	4	0x1	SMBWSTWENR5	Write enable assertion delay control register for memory bank 5. See Table 3-6 on page 3-12.

Table 3-1 PrimeCell SMC register summary (continued)

Address	Type	Width	Reset value	Name	Description
SMC RegBase + 0xA0	Read/write	8	0x80	SMBCR5	Control register for memory bank 5. See Table 3-8 on page 3-13.
SMC RegBase + 0xA4	Read/write	3	0x0	SMBSR5	Status register for memory bank 5. See Table 3-9 on page 3-15.
SMC RegBase + 0xA8	Read/write	4	0xF	SMBIDCYR6	Idle cycle control register for memory bank 6. See Table 3-2 on page 3-9.
SMC RegBase + 0xAC	Read/write	5	0x1F	SMBWST1R6	Wait state 1 control register for memory bank 6. See Table 3-3 on page 3-10.
SMC RegBase + 0xB0	Read/write	5	0x1F	SMBWST2R6	Wait state 2 control register for memory bank 6. See Table 3-4 on page 3-11.
SMC RegBase + 0xB4	Read/write	4	0x0	SMBWSTOENR6	Output enable assertion delay control register for memory bank 6. See Table 3-5 on page 3-11.
SMC RegBase + 0xB8	Read/write	4	0x1	SMBWSTWENR6	Write enable assertion delay control register for memory bank 6. See Table 3-6 on page 3-12.
SMC RegBase + 0xBC	Read/write	8	0x40	SMBCR6	Control register for memory bank 6. See Table 3-8 on page 3-13.
SMC RegBase + 0xC0	Read/write	3	0x0	SMBSR6	Status register for memory bank 6. See Table 3-9 on page 3-15.
SMC RegBase + 0xC4	Read/write	4	0xF	SMBIDCYR7	Idle cycle control register for memory bank 7. See Table 3-2 on page 3-9.
SMC RegBase + 0xC8	Read/write	5	0x1F	SMBWST1R7	Wait state 1 control register for memory bank 7. See Table 3-3 on page 3-10.

Table 3-1 PrimeCell SMC register summary (continued)

Address	Type	Width	Reset value	Name	Description
SMC RegBase + 0xCC	Read/write	5	0x1F	SMBWST2R7	Wait state 2 control register for memory bank 7. See Table 3-4 on page 3-11.
SMC RegBase + 0xD0	Read/write	4	0x0	SMBWSTOENR7	Output enable assertion delay control register for memory bank 7. See Table 3-5 on page 3-11.
SMC RegBase + 0xD4	Read/write	4	0x1	SMBWSTWENR7	Write enable assertion delay control register for memory bank 7. See Table 3-6 on page 3-12.
SMC RegBase + 0xD8	Read/write	8	0x00	SMBCR7	Control register for memory bank 7. See Table 3-8 on page 3-13.
SMC RegBase + 0xDC	Read/write	3	0x0	SMBSR7	Status register for memory bank 7. See Table 3-9 on page 3-15.
SMC RegBase + 0xE0	Read only	1	0x0	SMBEWS	External Wait Status register for all memory banks. See Table 3-10 on page 3-16.
SMC RegBase + 0xFE0	Read only	8	0x92	SMCPeriphID0	Peripheral ID register bits [7:0]. See Table 3-11 on page 3-18.
SMC RegBase + 0xFE4	Read only	8	0x10	SMCPeriphID1	Peripheral ID register bits [15:8]. See Table 3-12 on page 3-18.
SMC RegBase + 0xFE8	Read only	8	0x04	SMCPeriphID2	Peripheral ID register bits [23:16]. See Table 3-13 on page 3-18.
SMC RegBase + 0xFEC	Read only	8	0x00	SMCPeriphID3	Peripheral ID register bits [31:24]. See Table 3-14 on page 3-19.
SMC RegBase + 0xFF0	Read only	8	0x0D	SMCPCellID0	PrimeCell ID register bits [7:0]. See Table 3-15 on page 3-20.

Table 3-1 PrimeCell SMC register summary (continued)

Address	Type	Width	Reset value	Name	Description
SMC RegBase + 0xFF4	Read only	8	0xF0	SMCPCellID1	PrimeCell ID register bits [15:8]. See Table 3-16 on page 3-20.
SMC RegBase + 0xFF8	Read only	8	0x05	SMCPCellID2	PrimeCell ID register bits [23:16]. See Table 3-17 on page 3-20.
SMC RegBase + 0xFFC	Read only	8	0xB1	SMCPCellID3	PrimeCell ID register bits [31:24]. See Table 3-18 on page 3-21.

SMC RegBase is the base address of the PrimeCell SMC bank configuration registers.

3.2.1 Bank idle cycle control registers SMBIDCYR0-7

The eight SMBIDCYRx registers are the PrimeCell SMC bank idle cycle control registers, that must be programmed for the configuration of the PrimeCell SMC memory banks 0 to 7. Each register is identical in structure. Table 3-2 shows the bit assignment of a SMBIDCYRx register.

Table 3-2 SMBIDCYRx register bits

Bits	Name	Type	Description
31:4	Reserved	-	Reserved, do not modify, read as zero, write as zero.
3:0	IDCY	Read/write	Idle or turn-around cycles. Defaults to 1111 at reset. This field controls the number of bus turn-around cycles added between read and write accesses, to prevent bus contention on the external memory data bus. Turn around time = IDCY x t_{HCLK}

———— **Note** —————
 t_{HCLK} = period of **HCLK**

3.2.2 Bank wait state 1 control registers SMBWST1R0-7

The eight SMBWST1Rx registers are the PrimeCell SMC bank wait state one control registers, that must be programmed for the configuration of the PrimeCell SMC memory banks 0 to 7. Each register is identical in structure. Table 3-3 shows the bit assignment of a SMBWST1Rx register.

Table 3-3 SMBWST1Rx register bits

Bits	Name	Type	Description
31:5	Reserved	-	Reserved, do not modify, read as zero, write as zero.
4:0	WST1	Read/write	Wait State 1. Defaults to 11111 at reset. For SRAM and ROM, the WST1 field controls the number of wait states for read accesses. For burst ROM, the WST1 field controls the number of wait states for the first read access only. Wait state time = WST1 x t _{HCLK}

———— **Note** ————
t_{HCLK} = period of HCLK

3.2.3 Bank wait state 2 control registers SMBWST2R0-7

The eight SMBWST2Rx registers are the PrimeCell SMC bank wait state two control registers, that must be programmed for the configuration of the PrimeCell SMC memory banks 0 to 7. Each register is identical in structure. Table 3-4 shows the bit assignment of a SMBWST2Rx register.

Table 3-4 SMBWST2Rx register bits

Bits	Name	Type	Description
31:5	Reserved	-	Reserved, do not modify, read as zero, write as zero.
4:0	WST2	Read/write	Wait state 2. Defaults to 11111 at reset. For SRAM, the WST2 field controls the number of wait states for write accesses, and the external wait assertion timing for writes This wait state time is $WST2 \times t_{HCLK}$ in the case of SRAM. For burst ROM, the WST2 field controls the number of wait states for the burst read accesses after the first read. This wait state time is $WST2 \times t_{HCLK}$ in the case of burst ROM. For ROM, WST2 does not apply to ROM devices.

Note

t_{HCLK} = period of **HCLK**

3.2.4 Bank output enable assertion delay control registers SMBWSTOENR0-7

The eight SMBWSTOENRx registers are the PrimeCell SMC bank output enable assertion delay control registers, that must be programmed for the configuration of the PrimeCell SMC memory banks 0 to 7. Each register is identical in structure. Table 3-5 shows the bit assignment of a SMBWSTOENRx register.

Table 3-5 SMBWSTOENRx register bits

Bits	Name	Type	Description
31:4	Reserved	-	Reserved, do not modify, read as zero, write as zero.
3:0	WSTOEN	Read/write	Output enable assertion delay from chip select assertion. Defaults to 0000 at reset.

3.2.5 Bank write enable assertion delay control registers SMBWSTWENR0-7

The eight SMBWSTWENRx registers are the PrimeCell SMC bank write enable assertion delay control registers, that must be programmed for the configuration of the PrimeCell SMC memory banks 0 to 7. Each register is identical in structure. Table 3-6 shows the bit assignment of a SMBWSTWENRx register.

Table 3-6 SMBWSTWENRx register bits

Bits	Name	Type	Description
31:4	Reserved	-	Reserved, do not modify, read as zero, write as zero.
3:0	WSTWEN	Read/write	Write enable assertion delay from chip select assertion. Defaults to 0000 at reset.

3.2.6 Bank control registers SMBCR0-7

The eight SMBCRx registers are the PrimeCell SMC bank control registers, that must be programmed for the configuration of the PrimeCell SMC memory banks 0 to 7. Each register is identical in structure. Table 3-7 shows the memory bank default external memory width at reset.

Table 3-7 PrimeCell SMC reset default memory width

PrimeCell SMC memory bank	Default memory width
Bank 0	32-bit
Bank 1	8-bit
Bank 2	16-bit
Bank 3	8-bit
Bank 4	32-bit
Bank 5	32-bit
Bank 6	16-bit
Bank 7	8-bit, or SMMWCS7[1:0] at reset

Note

If the system boots from a Bank7 external ROM device, then the memory width for that bank must be configured using the external control pins **SMMWCS7[1:0]** to provide the correct default external memory width at reset.

Table 3-8 shows the bit assignment of a SMBCRx register.

Table 3-8 SMBCRx register bits

Bits	Name	Type	Description
31:8	Reserved	-	Reserved, do not modify, read as zero, write as zero.
7:6	MW	Read/write	Memory Width. Defaults to different values at reset for each bank, see Table 3-7. 00 - 8-bit 01 - 16-bit 10 - 32-bit 11 - reserved.

Table 3-8 SMBCRx register bits (continued)

Bits	Name	Type	Description
5	BM	Read/write	Burst mode: 0 - nonburst memory devices (default at reset) 1 - burst ROM memory.
4	WP	Read/write	Write protect: 0 - no write protection, for example, SRAM or write enabled Flash (default at reset). 1 - device is write protected, for example, ROM, burst ROM, read only Flash, or SRAM.
3	CSPol	Read/write	Chip select polarity bit indication for each bank: ^a 0 = active LOW SMCS (default at reset) 1 = active HIGH SMCS .
2	WaitEn	Read/write	External memory controller wait signal enable 0 = the PrimeCell SMC is not controlled by the external wait signal (default at reset). 1 = the PrimeCell SMC looks for the external wait input signal, SMWAIT .
1	WaitPol	Read/write	Polarity of the external wait input for activation 0 = the SMWAIT signal is active LOW (default at reset). 1 = the SMWAIT signal is active HIGH.
0	RBLE	Read/write	Read byte lane enable: 0 = nSMBLS [3:0] all deasserted HIGH during system reads from external memory. This is for 8-bit devices where the byte lane enable is connected to the write enable pin so must be deasserted during a read (default at reset). The nSMBLS signals are used as write enables in this configuration. 1 = nSMBLS [3:0] all asserted LOW during system reads from external memory. This is for 16 or 32-bit devices where the separate write enable signal is used and the byte lane selects must be held asserted during a read. The nSMWEN signal is used as the write enable in this configuration.

a. You must set this bit to the correct value if the device has the output enable lines tied off so that the outputs are always driven. If the incorrect polarity chip select is used the device can continuously drive data out.

3.2.7 Bank status registers SMBSR0-7

The eight SMBSRx registers are the PrimeCell SMC bank status registers. These registers indicate the state of various conditions, for example:

- errors on write protected regions
- time outs due to external waits
- any bus transfer errors.

The software can clear each of the error conditions by writing a 1 to the appropriate bit position. Table 3-9 shows the bit assignment of a SMBSRx register.

Table 3-9 SMBSRx register bits

Bits	Name	Type	Description
31:3	Reserved	-	Reserved, do not modify, read as zero, write as zero.
2	WaitToutErr	Read/write	External wait timeout error flag, read: 0 = no error (default at reset) 1 = external wait timeout error. Writing a 1 to this bit clears the wait timeout error status flag. Writing a 0 to this bit has no effect.
1	WriteProtErr	Read/write	Write protect error status flag, read: 0 = no error (default at reset) 1 = write protect error. Writing a 1 to this bit clears the write protect error status flag. Writing a 0 to this bit has no effect.
0	HSizeErr	Read/write	Bus transfer size error status flag, read: 0 = no error (default at reset) 1 = bus transfer size error Writing a 1 to this bit clears the bus transfer size error status flag. Writing a 0 to this bit has no effect.

3.2.8 External wait status register, SMBEWS

The SMBEWS register shows the current status of the external wait during an externally waited transfer. Table 3-10 shows the bit assignment of the SMBEWS register.

Table 3-10 SMBEWS register bits

Bits	Name	Type	Description
31:1	Reserved	-	Reserved, do not modify, read as zero, write as zero.
0	WaitStatus	Read	External wait status, read: 0 = SMWAIT deasserted 1 = SMWAIT asserted. After an externally waited transfer that was terminated early, this bit value can be used to detect when SMWAIT is deasserted. At all other times this bit reads zero.

3.2.9 Peripheral identification registers, SMCPeriphID0-3

The SMCPeriphID0-3 registers are four 8-bit registers, that span address locations 0xFE0-0xFEC. The registers can conceptually be treated as a single 32-bit register. The read-only registers provide the following options of the peripheral:

Part number [11:0] This is used to identify the peripheral. The three digit product code 092 is used for the PrimeCell SMC.

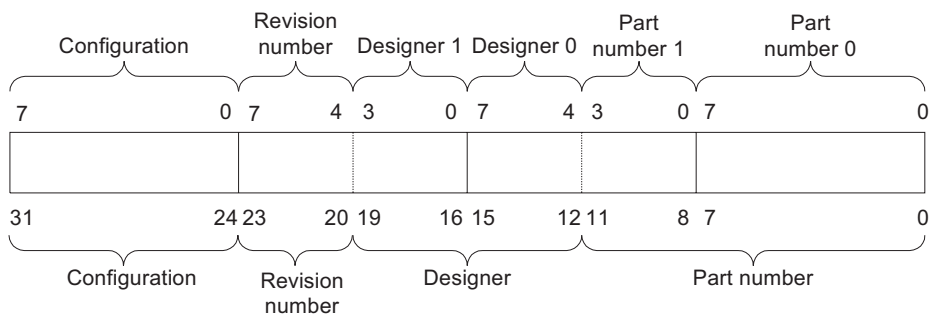
Designer [19:12] This is the identification of the designer. ARM Ltd. is 0x41 (ASCII A).

Revision number [23:20]
This is the revision number of the peripheral. The revision number starts from 0.

Configuration [31:24]
This is the configuration option of the peripheral. The configuration value is 0.

Figure 3-1 shows the bit assignment for the SMCPeriphID0-3 registers.

Actual register bit assignment



Conceptual register bit assignment

Figure 3-1 Peripheral identification register bit assignment

Note

When you design a systems memory map you must remember that the register has a 4KB memory footprint.

The 4-bit revision number is implemented by instantiating a component called *RevisionAnd* four times with its inputs tied off as appropriate, and the output sent to the read multiplexor.

All memory accesses to the peripheral identification registers must be 32-bit, using the LDR and STR instructions.

The four, 8-bit peripheral identification registers are described in the following subsections:

- *SMCPeriphID0* register on page 3-18
- *SMCPeriphID1* register on page 3-18
- *SMCPeriphID2* register on page 3-18
- *SMCPeriphID3* register on page 3-19.

SMCPeriphID0 register

The SMCPeriphID0 register is hard coded and the fields within the registers determine the reset value. Table 3-11 shows the bit assignment of the SMCPeriphID0 register.

Table 3-11 SMCPeriphID0 register

Bits	Name	Type	Function
31:8	-	-	Reserved, read undefined, must be written as zeros
7:0	Partnumber0	Read	These bits read back as 0x92

SMCPeriphID1 register

The SMCPeriphID1 register is hard coded and the fields within the registers determine the reset value. Table 3-12 shows the bit assignment of the SMCPeriphID1 register.

Table 3-12 SMCPeriphID1 register

Bits	Name	Type	Function
31:8	-	-	Reserved, read undefined, must be written as zeros
7:4	Designer0	Read	These bits read back as 0x0
3:0	Partnumber1	Read	These bits read back as 0x1

SMCPeriphID2 register

The SMCPeriphID2 register is hard coded and the fields within the registers determine the reset value. Table 3-13 shows the bit assignment of the SMCPeriphID2 register.

Table 3-13 SMCPeriphID2 register

Bits	Name	Type	Function
31:8	-	-	Reserved, read undefined, must be written as zeros
7:4	Revision	Read	These bits read back as 0x0
3:0	Designer1	Read	These bits read back as 0x4

SMCPeriphID3 register

The SMCPeriphID3 register is hard coded and the fields within the registers determine the reset value. Table 3-14 shows the bit assignment of the SMCPeriphID3 register.

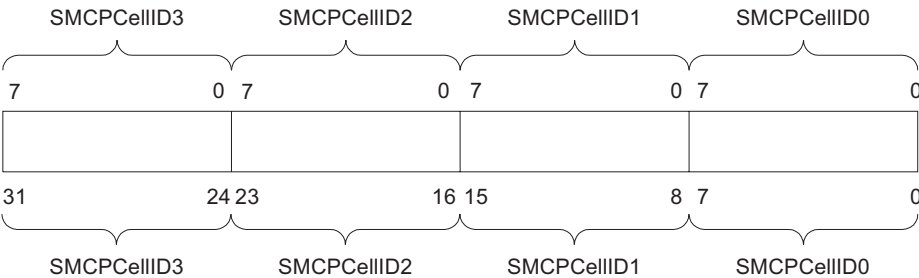
Table 3-14 SMCPeriphID3 register

Bits	Name	Type	Function
31:8	-	-	Reserved, read undefined, must be written as zeros
7:0	Configuration	Read	These bits read back as 0x0

3.2.10 PrimeCell identification registers, SMCPCellID0-3

The SMCPCellID0-3 registers are four 8-bit registers, that span address locations 0xFF0-0xFFC. The read-only registers can conceptually be treated as a single 32-bit register. The register is used as a standard cross-peripheral identification system. Figure 3-2 shows the bit assignment for the SMCPCellID0-3 registers.

Actual register bit assignment



Conceptual register bit assignment

Figure 3-2 PrimeCell identification register bit assignment

The four, 8-bit registers are described in the following subsections:

- *SMCPCellID0 register*
- *SMCPCellID1 register* on page 3-20
- *SMCPCellID2 register* on page 3-20
- *SMCPCellID3 register* on page 3-21.

SMCPCellID0 register

The SMCPCellID0 register is hard coded and the fields within the registers determine the reset value. Table 3-15 shows the bit assignment of the SMCPCellID0 register.

Table 3-15 SMCPCellID0 register

Bits	Name	Type	Function
31:8	-	-	Reserved, read undefined, must be written as zeros
7:0	SMCPCellID0	Read	These bits read back as 0x0D

SMCPCellID1 register

The SMCPCellID1 register is hard coded and the fields within the registers determine the reset value. Table 3-16 shows the bit assignment of the SMCPCellID1 register.

Table 3-16 SMCPCellID1 register

Bits	Name	Type	Function
31:8	-	-	Reserved, read undefined, must be written as zeros
7:0	SMCPCellID1	Read	These bits read back as 0xF0

SMCPCellID2 register

The SMCPCellID2 register is hard coded and the fields within the registers determine the reset value. Table 3-17 shows the bit assignment of the SMCPCellID2 register.

Table 3-17 SMCPCellID2 register

Bits	Name	Type	Function
31:8	-	-	Reserved, read undefined, must be written as zeros
7:0	SMCPCellID2	Read	These bits read back as 0x05

SMCPCellID3 register

The SMCPCellID3 register is hard coded and the fields within the registers determine the reset value. Table 3-18 shows the bit assignment of the SMCPCellID3 register.

Table 3-18 SMCPCellID3 register

Bits	Name	Type	Function
31:8	-	-	Reserved, read undefined, must be written as zeros
7:0	SMCPCellID3	Read	These bits read back as 0xB1

Chapter 4

Programmer's Model for Test

This chapter describes the registers of the ARM PrimeCell Static Memory Controller (PL092) and provides details required when programming the microcontroller. It contains the following:

- *Scan testing* on page 4-2.

4.1 Scan testing

The PrimeCell SMC has been designed to simplify:

- the insertion of scan test cells
- the use of *Automatic Test Pattern Generation* (ATPG).

This is the recommended method of manufacturing test.

Appendix A

ARM PrimeCell Static Memory Controller (PL092) Signal Descriptions

This chapter describes the AMBA AHB and module-specific non-AMBA AHB signals used with the ARM PrimeCell SMC. It contains the following sections:

- *AMBA AHB interface signals* on page A-2
- *AMBA AHB slave interface signals* on page A-3
- *AMBA AHB master interface signals* on page A-5
- *Internal signals* on page A-7
- *Input/output pad signals* on page A-9.

A.1 AMBA AHB interface signals

Table A-1 describes the common AMBA AHB signals.

Table A-1 Common AMBA AHB signals

Signal name	Type	Source/ destination	Description
nHCLK	Input	Clock control	Inverted bus clock input. It is used only for the generation of the write enable output.
HCLK	Input	Clock control	Bus clock input, that times all bus transfers. All signal timings are related to the rising edge.
HRESETn	Input	Reset controller	Bus reset input, active LOW, used to reset the system and the bus when asserted LOW.
HREADYIN	Input	Other AHB slaves	Transfer completed input. When HIGH the HREADYIN signal indicates that a transfer has finished on the bus. This signal can be driven LOW to extend a transfer.

A.2 AMBA AHB slave interface signals

Table A-2 describes the AMBA AHB slave interface signals.

Table A-2 AMBA AHB slave interface signals

Signal name	Type	Source/ destination	Description
HADDR[28:0]	Input	AMBA AHB master	System address bus, least significant 29 bits, driven by the active bus master.
HTRANS[1:0]	Input	AMBA AHB master	Indicates the type of the current transfer, which can be NONSEQUENTIAL, SEQUENTIAL, IDLE, or BUSY.
HWRITE	Input	AMBA AHB master	Transfer direction signal. When HIGH, this signal indicates a write to the PrimeCell AHB SMC and when LOW, a read from the PrimeCell AHB SMC block.
HSIZE[2:0]	Input	AMBA AHB master	Transfer size signal. This signal indicates the size of the current transfer, which can be byte (8-bit), halfword (16-bit), or word (32-bit).
HBURST[2:0]	Input	AMBA AHB master	Indicates if the transfer forms part of a burst. Four, eight, and 16 beat bursts are supported and the burst can either be incrementing or wrapping.
HWDATA[31:0]	Input	AMBA AHB master	Write data bus, used to transfer data to this block.
HSELSMC	Input	AMBA AHB decoder	Slave select signal for PrimeCell AHB SMC memory banks.
HSELREG	Input	AMBA AHB decoder	Slave select signal for PrimeCell AHB SMC control and status registers.

Table A-2 AMBA AHB slave interface signals (continued)

Signal name	Type	Source/ destination	Description
HRDATA[31:0]	Output	AMBA AHB slave	Read data bus, used to read data from this block.
HREADYOUT	Output	AMBA AHB slave	Transfer done output. When HIGH indicates that a transfer has finished on the bus. This signal can be driven LOW to extend a transfer.
HRESP[1:0]	Output	AMBA AHB slave	The transfer response provides additional information on the status of a transfer. Two different responses are provided, OKAY and ERROR.

A.3 AMBA AHB master interface signals

Table A-3 describes the AMBA AHB master interface signals.

Table A-3 AMBA AHB master interface signals

Signal name	Type	Source/ destination	Description
HRESPTIC[1:0]	Input	AMBA AHB slave	The transfer response provides additional information on the status of a transfer. The TIC supports both SPLIT and RETRY responses.
HRDATATIC[31:0]	Input	AMBA AHB slave	The read data bus is used to transfer data from bus slaves to the bus master during test read operations.
HGRANTTIC	Input	AMBA AHB arbiter	This signal indicates that the TIC is currently the highest priority master. Ownership of the address or control signals changes at the end of the transfer when HREADYIN is HIGH.
HADDRTIC[31:0]	Output	AMBA AHB slave	The 32-bit system address bus.
HTRANSTIC[1:0]	Output	AMBA AHB slave	Indicates the type of the current transfer, which can be NONSEQUENTIAL, SEQUENTIAL, or IDLE. The TIC does not use the BUSY transfer type.
HWRITETIC	Output	AMBA AHB slave	Transfer direction signal. When HIGH, this signal indicates a write to the PrimeCell AHB SMC and when LOW, a read from the PrimeCell AHB SMC block.
HSIZETIC[2:0]	Output	AMBA AHB slave	Transfer size signal. This signal indicates the size of the current transfer, which can be byte (8-bit), halfword (16-bit) or word (32-bit). The TIC does not support larger transfer sizes.

Table A-3 AMBA AHB master interface signals (continued)

Signal name	Type	Source/ destination	Description
HBURSTTIC[2:0]	Output	AMBA AHB slave	Indicates if the transfer forms part of a burst. The TIC always performs incrementing bursts of unspecified length.
HPROTTIC[3:0]	Output	AMBA AHB slave	The protection control signals indicate if the transfer is an opcode fetch or data access, as well as if the transfer is a Supervisor mode access or User mode access. These signals can also indicate whether the current access is cacheable or unbufferable.
HWDATATIC[31:0]	Output	AMBA AHB slave	The write data bus is used to transfer data from the master to bus slaves during write operations. A minimum data bus width of 32 bits is recommended, however this can easily be extended to allow for higher bandwidth operation.
HBUSREQTIC	Output	AMBA AHB arbiter	A signal from the TIC to the bus arbiter that indicates that it requires the bus.
HLOCKTIC	Output	AMBA AHB arbiter	When HIGH this signal indicates that the TIC requires locked access to the bus and no other master must be granted the bus until this signal is LOW.

A.4 Internal signals

Table A-4 describes the internal signals.

Table A-4 Internal signal descriptions

Signal name	Type	Source/ destination	Description
BIGENDIAN	Input	System	This static configuration bit indicates the type of endianness of the memory system: 0 = little-endian 1 = big-endian.
EXTBUSMUX	Input	System	This static configuration bit indicates if the internal bus multiplexer (DBI) is used, or if an external bus multiplexer (for example EBI) is used instead: 0 = internal bus multiplexer 1 = external bus multiplexer
MCBUSREQ	Input	Additional memory controller	Bus request signal, indicates that the additional memory controller block has requested use of the external bus.
MCADDR[25:0]	Input	Additional memory controller	Address output to external bus.
MCDATAOUT[31:0]	Input	Additional memory controller	Data output, used to write data from the additional memory controller to the external data bus.
MCDATAEN[3:0]	Input	Additional memory controller	Tristate input/output pad enable for the byte lanes of the external memory data bus MCDATAOUT[31:0] . Enables the byte lanes [31:24], [23:16], [15:8], and [7:0] of the data bus independently.
MCBUSGNT	Output	Additional memory controller	Bus grant signal, indicates that the additional memory controller is granted control of the external bus.
SMBUSREQEBI	Output	External bus multiplexer	Bus request signal, indicates that the SMC has requested use of the external bus. Only used when EXTBUSMUX tied to one.

Table A-4 Internal signal descriptions (continued)

Signal name	Type	Source/ destination	Description
SMBUSGNTebi	Input	External bus multiplexer	Bus grant signal, indicates that the SMC is granted control of the external bus. Only used when EXTBUSMUX tied to one.
TICBUSREQebi	Output	External bus multiplexer	Bus request signal, indicates that the TIC has requested use of the external bus. Only used when EXTBUSMUX tied to one.
TICBUSGNTebi	Input	External bus multiplexer	Bus grant signal, indicates that the TIC is granted control of the external bus. Only used when EXTBUSMUX tied to one.
TICREADEbi	Output	External bus multiplexer	Data bus enable for TBUSOUTebi . Only used when EXTBUSMUX tied to one.
TBUSOUTebi[31:0]	Output	External bus multiplexer	TIC data bus output used during reads from the external tester. Only used when EXTBUSMUX tied to one.
REMAP	Input	System	Indicates the state of the memory map: 0 = reset memory map (SMCS7 mapped to SMCS0) 1 = normal memory map.
SCANENABLE	Input	System	Dummy pin that is used as a dedicated scan enable input.
SCANINnHCLK	Input	System	Dummy pin that is used as a dedicated nHCLK scan chain input.
SCANINHCLK	Input	System	Dummy pin that is used as a dedicated HCLK scan chain input.
SCANOUTnHCLK	Output	System	Dummy pin that is used as a dedicated nHCLK scan chain input.
SCANOUTHCLK	Output	System	Dummy pin that is used as a dedicated HCLK scan chain input.

A.5 Input/output pad signals

Table A-5 describes the signals to the input/output pads.

Table A-5 Input/output pad signals

Signal name	Type	Source/ destination	Description
SMMWCS7[1:0]	Input	Input pad	These static configuration bits indicate the memory width used for boot memory bank one: 00 = 8-bit 01 = 16-bit 10 = 32-bit 11 = no change.
SMWAIT	Input	Input pad	Wait mode input from external memory controller. Active HIGH, or active LOW, (default), as programmed in the SMC control registers for each bank.
CANCELSMWAIT	Input	Input pad	This signal allows the system to recover from an externally waited transfer that takes longer than expected to finish. Active HIGH.
SMDATAIN[31:0]	Input	Input pad	External input data bus used to read data from memory bank.
SMDATAOUT[31:0]	Input	Input pad	External output data used to write data from SMC to memory bank.
SMADDR[25:0]	Output	Output pad	External memory address bus, to external memory banks.
SMCS[7:0]	Output	Output pad	Chip select for external memory banks 7 to 0, default active LOW.
nSMDATAEN[3:0]	Output	Output pad	Tristate input/output pad enable for the byte lanes of the external memory data bus SMDATA[31:0] , active LOW. Enables the byte lanes [31:24], [23:16], [15:8], and [7:0] of the data bus independently.
nSMWEN	Output	Output pad	Write enable for the external memory banks, active LOW.

Table A-5 Input/output pad signals (continued)

Signal name	Type	Source/ destination	Description
nSMBLS[3:0]	Output	Output pad	Byte lane select signals, active LOW. The signals nSMBLS[3:0] select byte lanes [31:24], [23:16], [15:8], [7:0] on the data bus.
nSMOEN	Output	Output pad	Output enable for external memory banks, active LOW.
TESTREQA	Input	Input pad	This is the Test Bus Request A input signal and is required as a dedicated device pin. During normal system operation the TESTREQA signal is used to request entry into the test mode. During test TESTREQA is used, in combination with TESTREQB , to indicate the type of test vector that is applied in the following cycle.
TESTREQB	Input	Input pad	During test this signal is used, in combination with TESTREQA , to indicate the type of test vector that is to be applied in the following cycle.
TESTACK	Output	Output pad	The test bus acknowledge signal gives external indication that the TIC is granted and also indicates when a test access is complete. When TESTACK is LOW, the current test vector must be extended until TESTACK becomes HIGH.

Index

The items in this index are listed in alphabetic order, with symbols and numerics appearing at the end. The references given are to page numbers.

A

Access sequencing 2-7
AMBA
 compatibility 1-2

B

Booting from ROM after reset 2-46
Burst ROM 2-16
Bus turnaround 2-26
Byte lane write control 2-33

D

Data bus interface 2-3

E

Endian support 2-2
External 2-10

External bank SMCS7 size
 configuration 2-47
External wait control 2-29

F

Flash memory 2-25

M

Memory bank select 2-6

N

Non-AMBA AHB signals A-7

P

PrimeCell AHB SMC
 features 1-2
 registers 3-3

PrimeCell SMC

 block diagram 2-3
 features 1-2
 I/O connections 1-6
 memory banks 2-2
 operation 2-6
 overview 2-2
Programmer's model 3-2

R

Registers

 SMBCR 0 - 7 3-13
 SMBEWS 3-16
 SMBSR 0 - 7 3-15
 SMBWST1R 0 - 7 3-10
 SMBWST2R 0 - 7 3-11
 SMBWSTOENR 0 - 7 3-11
 SMBWSTWENR 0 - 7 3-12
 SMBCBR 0 - 7 3-9
 SMCPCellID0-3 3-19
 SMCPeriphID0-3 3-16
 summary 3-3
ROM, SRAM and FLASH 2-9

S

Signals

- AMBA AHB interface A-2
- AMBA AHB master interface A-5
- AMBA AHB slave interface A-3
- Internal A-7

SMC core 2-3

- AMBA AHB interface 2-5
- External bus interface 2-5
- Transfer control 2-5

SMWAIT

- assertion timing 2-29
- deassertion timing 2-30
- timing diagrams 2-30

Static memory

- write control 2-19

Static memory read control 2-9

T

Test interface controller 2-3

W

Wait state generation 2-8

Write protection 2-8

Write timing for Flash 2-25